

Graph Neural Networks for Integrated Circuit Design, Reliability, and Security: Survey and Tool

ZIAD EL SAYED, University College London, London, United Kingdom

ZENG WANG, Electrical and Computer Engineering, New York University, Brooklyn, United States

HANA SELMANI, New York University Abu Dhabi, Abu Dhabi, United Arab Emirates

JOHANN KNECHTEL, New York University Abu Dhabi, Abu Dhabi, United Arab Emirates

OZGUR SINANOGLU, New York University Abu Dhabi, Abu Dhabi, United Arab Emirates

LILAS ALRAHIS, Computer and Information Engineering, Khalifa University, Abu Dhabi, United Arab Emirates

Graph neural networks (GNNs) have significantly advanced learning and predictive tasks in many domains like social networks and biology. Given the inherent graph structure of integrated circuits (ICs), GNNs have also shown strong results for various IC-related tasks. Here, we review GNN methodologies across three key areas for ICs: electronic design automation (EDA), reliability, and hardware security. We introduce a comprehensive taxonomy and survey, covering various tasks and their solutions by GNNs in depth. We also outline key challenges like scalability and EDA tool integration. Finally, we present GNN4CIRCUITS, an open-source tool for plug-and-play GNN integration for various IC tasks.

CCS Concepts: • **Security and privacy** → **Hardware reverse engineering**; **Hardware security implementation**; • **Hardware** → **Electronic design automation**; **Hardware reliability**; **Hardware test**; • **Computing methodologies** → **Machine learning**; **Neural networks**

Additional Key Words and Phrases: Hardware security, hardware reliability, graph neural networks, survey, customizable tool

ACM Reference Format:

Ziad El Sayed, Zeng Wang, Hana Selmani, Johann Knechtel, Ozgur Sinanoglu, and Lilas Alrahis. 2025. Graph Neural Networks for Integrated Circuit Design, Reliability, and Security: Survey and Tool. *ACM Comput. Surv.* 58, 4, Article 102 (October 2025), 44 pages. <https://doi.org/10.1145/3769081>

Ziad El Sayed and Zeng Wang contributed equally to this research.

Ziad El Sayed and Lilas Alrahis were with New York University Abu Dhabi.

Authors' Contact Information: Ziad El Sayed, University College London, London, United Kingdom; e-mail: ziad.sayed.24@ucl.ac.uk; Zeng Wang, Electrical and Computer Engineering, New York University, Brooklyn, NY, USA; e-mail: zw3464@nyu.edu; Hana Selmani, New York University Abu Dhabi, Abu Dhabi, United Arab Emirates; e-mail: hs4311@nyu.edu; Johann Knechtel, New York University Abu Dhabi, Abu Dhabi, United Arab Emirates; e-mail: johann@nyu.edu; Ozgur Sinanoglu, New York University Abu Dhabi, Abu Dhabi, United Arab Emirates; e-mail: ozgursin@nyu.edu; Lilas Alrahis, Computer and Information Engineering, Khalifa University, Abu Dhabi, United Arab Emirates; e-mail: lilas.malrahis@ku.ac.ae.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 0360-0300/2025/10-ART102

<https://doi.org/10.1145/3769081>

1 Introduction

The increasing complexity of **integrated circuit (IC)** design and manufacturing, along with recent advancements in **machine learning (ML)**, has prompted researchers to explore various ML techniques to optimize different aspects of IC design. As a result, ML has **outperformed state-of-the-art (SOTA)** methods in addressing numerous challenges in **electronic design automation (EDA)**, including hardware security and reliability [59, 79]. For example, ML has shown success in identifying **hardware Trojans (HTs)** [81] and forecasting hardware malfunctions [92], among other significant applications. Among the diverse ML approaches studied, **graph neural networks (GNNs)** have shown considerable promise in transforming IC design processes. This is because circuits can naturally be modeled as graphs, whether provided in layout format, transistor-level netlist, **gate-level netlist (GLN)**, or **register-transfer level (RTL)**. As a result, many EDA tasks can be efficiently framed as graph-based problems that are directly applicable to GNNs [17].

With the rapid advancement of GNNs, the past few years have seen an exponential increase in research interest focused on utilizing GNNs to tackle a variety of IC-related challenges, as shown in Figure 1. For example, the number of publications related to GNNs in IC design increased significantly from merely 7 in 2018 to 1,553 by 2023. The concept of GNNs is relatively new, as reflected by the growth in publications from 50 in 2018 to 8,402 in 2023. Given the plethora of options for circuit-to-graph conversion, extraction, and GNN architectures, each tailored to different tasks, it is crucial to summarize the progress made in this area and highlight directions for future research.

Several surveys have been conducted on generic GNNs as well as those tailored to specific tasks. For example, the authors in [206] explore the applications of GNNs in data mining, discussing their usage across multiple domains. References [84, 199, 205] further examine the versatility of GNNs. Additionally, specific surveys, such as the one in [141], provide a systematic review of GNN-based methods for stock market forecasting. Clearly, there has been significant interest in GNNs across a wide range of fields, making comprehensive surveys essential for researchers seeking to navigate and synthesize the growing body of work. Nevertheless, GNN-related surveys in IC research remain limited, often focusing on narrow aspects.¹ A comprehensive survey on the use of GNNs for different IC-related tasks and fields is needed, as a broad understanding of various applications is key to selecting suitable GNN architectures for circuit learning, determining appropriate graph structures and features, and effectively applying GNNs to downstream IC-related tasks.

1.1 Our Contributions

We provide the first comprehensive survey of GNN applications for advancing IC design across the following key domains: EDA in general, reliability of modern ICs, and hardware security. We build upon our previous survey [17], significantly expanding our coverage to include a broader range of

¹References [79, 91, 131, 154] provide surveys on the utilization of ML to advance EDA in general, only briefly touching on GNNs if at all. References [121, 123, 126, 157] are recent surveys dedicated specifically to the topic of GNN usage in EDA, without considering aspects such as security or reliability. Furthermore, the work in [204] surveys the application of GNNs for predicting **high-level synthesis (HLS)** performance. The studies in [119, 172] explore the general use of ML in the field of hardware security, while the review in [21] provides a comprehensive overview of the specific use of GNNs.

²The search filters applied for general applications were: (“GNN” or “gnn”) and “Graph Neural Network”. For circuit-related contexts, the filters were more specific, including more terms (“Hardware Security” or “Hardware Reliability” or “EDA” or “Electronic Design Automation” or “Circuit Design” or “chip design” or “hardware implementation” or “hardware vulnerability”). Sources relate to academic journals and conference materials accessed through the EBSCO search engine [58].

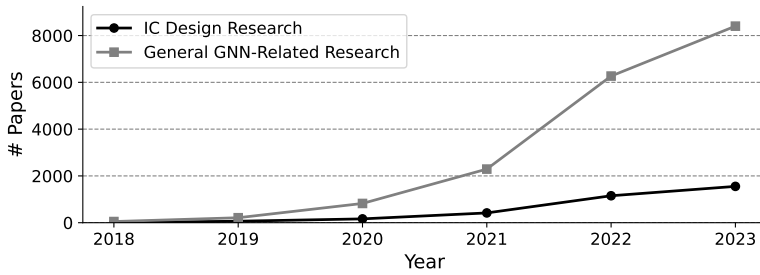


Fig. 1. The exponential growth in GNN-related research for both circuit-specific and general applications. ²

critical topics and discussing more than 175 papers. We also present and release a versatile and configurable circuit-to-graph conversion tool that accompanies our work and provides researchers with a method to extend our surveyed domain. Furthermore, we provide a clear introduction to the fundamentals of both graph-based learning and IC-related applications, making this article accessible to new readers. Our contributions are summarized as follows.

- (1) We introduce a **general pipeline and taxonomy for employing GNNs** to advance IC design, focusing on (i) GNNs for general EDA and IC design, (ii) GNNs for improving IC design reliability, and (iii) GNNs for secure IC design and analysis.
- (2) We present a **thorough survey** and offer a guided tour through the literature, providing in-depth analysis of SOTA GNN tasks, architectures, graph types, circuit design levels, extracted features, and more. Accompanying this survey, we have gathered resources on GNNs for IC design, encompassing a range of applications, models, datasets, and open-source code, and consolidated them in our **GNN4IC hub** [<https://github.com/DfX-NYUAD/GNN4IC>].
- (3) Existing works on using GNNs for circuit analysis often employ specific implementations tailored to their respective problem domains. While many of these codes are open-sourced, they are typically designed to handle specific inputs, such as circuit types, standard-cell libraries, and specific GNN architectures. To address this limitation, we introduce the **GNN4CIRCUITS tool**, an accessible and customizable Python-based tool for leveraging GNNs in circuit analysis. The tool supports various circuit types as input, enables the extraction of diverse features, facilitates the construction of different graph types, and accommodates various GNN architectures. This plug-and-play capability offers researchers flexibility and aims to expedite research in this domain. The tool is released at [<https://github.com/DfX-NYUAD/GNN4CIRCUITS>].

1.2 Paper Organization

The structure of the rest of this article is outlined as follows. Section 2 overviews the background of IC design and GNNs, lists IC-related notations and defines graph-related concepts. Section 3 presents a taxonomy for applying GNNs to solve IC-related tasks, including EDA, hardware reliability, and hardware security. Section 4 outlines future directions in the application of GNNs to IC-related problems, drawn from the lessons learned from our thorough review. Section 5 describes the GNN4CIRCUITS tool. Finally, Section 6 summarizes the article and recaps future directions for using GNNs in IC-related domains.

2 Background

In this section, we provide an overview of EDA, hardware reliability, and security in the context of IC design. We also provide thorough background on GNNs, detail the common steps/notations employed in these fields, and define associated concepts.

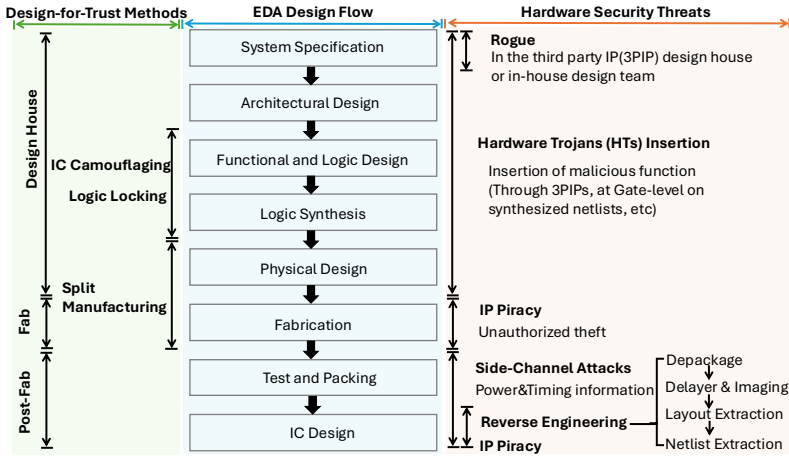


Fig. 2. The general flow of IC design [121] with its involved hardware security threats [30, 89] and **design-for-trust (DfTr)** methods [168].

2.1 Electronic Design Automation (EDA)

The importance of EDA tools becomes evident as IC designers leverage progress in microelectronics while simultaneously handling growing design complexities [87]. Over the past few years, ML has evolved to play a central role in advancing next-generation EDA tools, as we present in this article. Yet, the fundamental structure of the overall EDA pipeline has largely been maintained [121]. This structure is illustrated below and in Figure 2.

System specification and architectural design. This stage defines the objectives and requirements related to functionality, performance, and physical implementation. It involves establishing clear goals for the system's operation and constraints. Modeling languages like SystemC are used to create formalized descriptions, allowing for precise and structured representation of the system's architecture and behavior [121].

Behavioral and logic design represent the early stages of digital circuit development, where the high-level functionality of a system is defined. In the behavioral design phase, the system's intended behavior is specified using high-level abstractions, often in **hardware description languages (HDLs)**. This phase focuses on defining how the system should perform, outlining the overall data flow, control logic, and operations without specifying exact hardware implementations.

Logic design follows, where the abstract behavior is translated into specific logic operations using combinational and sequential circuits. During this stage, the focus is on creating the actual logic structure—designing the arithmetic units, multiplexers, state machines, and other components necessary to achieve the desired functionality. The outcome of logic design is a detailed blueprint of how the circuit will implement the behavior defined in the earlier phase. This step was traditionally manual, but nowadays it is also well supported by EDA tools offering HLS [193].

The integration of third-party **intellectual property (IP)** cores is part of the process. Third-party IPs, such as pre-designed processors, memory blocks, or communication interfaces, can be incorporated into the design to enhance functionality and reduce development time.

Logic synthesis is the process of converting a high-level hardware description, typically written in Verilog or VHDL, into a GLN composed of basic logic gates like AND, OR, and INV. This stage translates the functional design into a form that can be physically implemented. To optimize the design, synthesis involves several tasks, including minimization of logic gates, power optimization, and meeting timing constraints. The result is a netlist that accurately reflects the intended func-

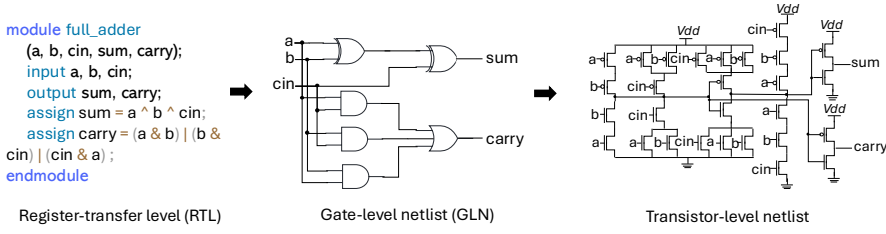


Fig. 3. Examples of different circuit representations. A standard full adder circuit is first represented at a high-level RTL using Verilog, then synthesized to a GLN, and finally to a transistor-level netlist.

tionality while adhering to design goals such as area, speed, and power. An example is shown in Figure 3, where a standard full adder circuit is represented in RTL using Verilog language first, and then synthesized to a GLN by realizing the function using standard Boolean gates.

Physical design. The GLN is transformed into the physical layout, which consists of components such as macros, memories, wires, and gates. Due to the complexity of this stage, it is typically divided into several key tasks: partitioning or floorplanning, power and ground network design, component placement, clock distribution, routing, and ensuring timing closure. Each step is crucial for translating the logical design into a manufacturable layout.

Verification and signoff. The physical layout must undergo rigorous verification to ensure compliance with a wide range of design and manufacturing rules, ensuring both functional correctness and desired electrical performance. After successfully meeting all specified criteria, the design is ready for signoff and can proceed to the tape-out stage, marking the final step before actual manufacturing [165].

2.2 Hardware Reliability

ICs are often deployed for long lifespans; therefore, ensuring hardware reliability is essential. Furthermore, the applications of ICs can be safety-critical or costly in the event of a failure. Below, we provide an overview of key IC-related reliability concerns and the methods to mitigate them.

2.2.1 Challenges. **Process variation** arises from imperfections in the IC manufacturing process, potentially leading to circuit timing violations, increased power consumption, or functional error. This time-independent effect differs from chip to chip and even within the same chip. Typical sources of variation in state-of-the-art FinFET devices include random dopant fluctuation, metal gate granularity, and line-edge roughness [228]. Accurately estimating process variation is essential for achieving high yield at performance.

Transistor aging. Transistor performance in a circuit can degrade over time, ultimately affecting the performance of the overall circuit. Transistor aging is influenced by physical phenomena such as **hot-carrier injection (HCI)** and **bias temperature instability (BTI)** [28]. Over time, electrical charges accumulate in the gate oxide of transistors, raising their threshold voltages. Additionally, interface traps at the *Si-SiO₂* junction can add more unwanted charges and further threshold voltage (V_{th}) increases. This results in slower transistor switching times and increased circuit path delays.

Environmental variation stems from run-time fluctuations like temperature changes and power supply noise that directly affect transistor electrical behavior [148]. These variations are most critical in high-performance ICs made with advanced technology nodes. Temperature variations alter transistor characteristics and speed differently across technology generations—newer technologies (45 nm and below) typically improve at higher temperatures, while older nodes slow down. Moreover, global and local power supply fluctuations shift transistor characteristics from nominal values

[104]. Unlike process variation or aging, these environmental changes are temporary and resolve when the device returns to nominal operating conditions.

2.2.2 Mitigation Strategies.

Timing error tolerance are essential to make circuit designs resilient to timing errors, especially for circuits operating at low voltages where the effect of delay uncertainty is pronounced. Unlike static process variations, dynamic variations manifest themselves in the field as a function of time and environment, which cannot be compensated by one-time pre-silicon and post-silicon tuning techniques [147]. Therefore, various timing error tolerance techniques have been developed for preventing, correcting, or avoiding timing errors, such as circuit topology optimization [32], temporal error masking [64, 107], and graceful quality degradation [35, 152].

Guardband, a traditional approach to aging mitigation, increases slack in the design critical path [4, 94]. This technique extends device lifetime by impacting clock frequency, requiring more severe performance degradation to surpass the extended clock period. Recent advancements include improved high-level aging models [173, 178] that calculate the average failures-in-time rate of the processor core and ML-based techniques [95, 98] that devise appropriately sized guardbands without unnecessary performance loss, thereby enabling effective aging mitigation.

Operating voltage can be increased to allow higher V_{th} degradation to operate without timing failures, thereby extending device lifetime [164]. However, this approach results in additional power consumption, causing higher operating temperatures that exacerbate most aging mechanisms, including negative-BTI, HCI, and time-dependent dielectric breakdown.

2.3 Hardware Security

The globalization of the IC manufacturing process involves multiple entities worldwide, leading to a complex, outsourced supply chain. This raises significant security concerns, including the implantation of malicious HTs, piracy of design IP, and **reverse-engineering (RE)** [151, 158]. These issues can result in financial losses for IP owners and pose risks to end-users.

We will now selectively review threats and countermeasures pertinent to the scope of this article, shown in Figure 2. For a broader analysis, references [99–101, 196] offer extensive insights and include discussions on advancing EDA for secure IC design.

2.3.1 Hardware Security Threats.

IP piracy refers to the unauthorized theft of the design IP by an adversary (such as foundry or end-user) to create competing devices without bearing the costs of research and development [99, 166]. Adversaries can fabricate stolen IPs, integrate them into devices, and profit from them—posing a serious concern to leading-edge design houses that invested heavily in the original design. Another form of piracy occurs when an untrusted foundry overproduces chips beyond authorized quantities and illegally sells excess chips [159, 160].

HTs are malicious modifications embedded within ICs by attackers to induce malignant outcome [158], such as the leakage of sensitive assets like crypto keys, or to disrupt design functionality [38, 90]. These HTs are characteristically stealthy, rarely activated, and can be integrated at various stages of the IC design flow. Common insertion points include adversarial third-party IP cores, compromised design tools, or malicious employees during chip fabrication [99].

RE is a process intended to uncover the IC/IP design, technology, or functionality by thoroughly inspecting each layer of the chip [135]. RE can be used for both offensive actions, either various attacks or defensive purposes, such as detecting IP infringement [187], verifying IP implementation [175], and identifying HTs [36].

Side-channel attacks exploit unintended physical leakage such as power consumption, electromagnetic emissions, or timing variations to extract sensitive data like cryptographic keys [153].

These attacks do not require altering the design; instead, they rely on analyzing the device's normal operation to infer hidden information. Techniques like differential and correlation power analysis correlate measured signals with internal operations [102].

Probing attacks on ICs refer to physical methods using invasive techniques, such as chip decapsulation and focused-ion-beam milling, to expose internal nodes and enable direct electrical measurements with microprobes or laser voltage probing [189]. High-resolution imaging can then map interconnects and reveal buried layers, bypassing logical defenses unless hardware shielding or active detection mechanisms are employed [189].

2.3.2 Design-for-Trust Methods. The global supply chain introduces multiple hardware security threats, such as IP piracy, RE and other attacks, driving the development of DfTr methods to mitigate these risks. Figure 2 shows the stages of the IC supply chain protected by each DfTr method discussed below.

Logic locking protects digital circuits from unauthorized access or tampering by embedding encryption into the design. This process involves adding key-gates to the circuit, which are special gates that require a secret key to function correctly. Key-gates are inserted into the circuit in such a way that the correct operation of the circuit depends on the correct key being provided. If the correct key is not supplied, the circuit will not perform its intended function, thereby preventing unauthorized use or RE. This method effectively secures IP by ensuring that only those with the correct key can fully utilize or understand the circuit's functionality. After the fabrication and testing process, the owner programs the secret key into a tamper-proof, on-chip memory. References [9, 12, 23, 24, 110, 170, 176] offer additional insights on logic locking.

Layout camouflaging modifies the visual appearance of a chip to hide the design IP, effectively obfuscating the design details at the transistor-level [144], interconnect-level [143], or gate-level [149], thereby safeguarding it from RE attacks.

Split manufacturing protects the design IP from untrusted foundries during fabrication [142, 150]. The design IP is split into **front-end-of-line (FEOL)** and **back-end-of-line (BEOL)**. Considered independently, the split portions become a "sea of logic gates" without any wiring (interconnect) information for the FEOL foundry, whereas they reduce to system-level wiring without any logic gate-related information for the BEOL foundry. Split manufacturing is beneficial since: (i) outsourcing of the FEOL requires access to advanced technology nodes, and (ii) BEOL fabrication is significantly less complex than FEOL fabrication.

2.4 Graph Neural Networks (GNNs)

Next, we introduce essential graph-related concepts, relate them to **artificial intelligence (AI)** and **deep learning (DL)**, offer an overview of GNNs, and outline commonly used notations (Table 1).³

2.4.1 From AI and DL to GNNs.

AI broadly refers to systems that perform tasks requiring human cognition, with DL employing multi-layered neural networks to learn from large datasets [108]. While traditional DL frameworks operate on grid-like data, such as images or sequences, graph-based data require specialized architectures. GNNs combine DL with graph-theoretic insights to learn relational representations [206], enabling applications in domains with inherently graph-structured data, such as social networks, biological systems, and circuits. Huang *et al.* [80] survey ML applications across the entire

³For reference, matrices are denoted using capital letters. For example, the adjacency matrix is represented as A . Vectors are denoted using lowercase bold letters; for instance, the feature vector of node v is denoted as $\mathbf{x}_v$. Scalar variables, such as those indicating sizes, are represented using lowercase letters, while sets are denoted using bold capital letters. We denote ML models using monospaced font (e.g., g). In addition, mathematical operations such as standard deviation (std), concatenation (concat), minimum (min), and other similar operations are also denoted using the monospaced font style.

Table 1. Commonly used Symbols and Notations

Notation	Description	Notation	Description
$\mathcal{G}$	A graph defined as $\mathcal{G} = (V, E)$	A	Adjacency matrix of $\mathcal{G}$
V	Node set of $\mathcal{G}$	E	Edge set of $\mathcal{G}$
u, v	Nodes in $\mathcal{G}$, i.e., $u, v \in V$	x_v	Initial feature vector of node v
n	Number of nodes in $\mathcal{G}$, $n = V $	k	Size of each node feature vector, $k = x_v $
$X \in \mathbb{R}^{n \times k}$	Node feature matrix	z_v	Embedding vector of node v for node-level tasks
Z	Node embedding matrix	$z_{\mathcal{G}}$	Graph-level embedding of $\mathcal{G}$ for graph-level tasks
$Z^{(0)}$	Initial node feature matrix X	$Z^{(l)}$	Node embedding matrix at l th iteration
L	Total number of message passing layers	$Z^{(L)}$	Final node embedding matrix
$W^{(l)}$	Weight matrix at l th iteration	σ	Non-linear activation function (e.g., ReLU)
$\hat{y}_v$	Prediction for node v : $\hat{y}_v = g(z_v)$	$\hat{y}_{\mathcal{G}}$	Prediction for graph $\mathcal{G}$: $\hat{y}_{\mathcal{G}} = g(z_{\mathcal{G}})$
y_v	Ground-truth label or value for node v	$\bar{y}, \tilde{y}$	Mean of ground-truth, predicted values
$\mathcal{N}(v)$	Neighbors of node v	$ \mathcal{N}(v) $	Degree of node v (including self-loops)
h	Hop size (e.g., 1-hop, 2-hop)	$\mathcal{G}_{(S,h)}$	h -hop subgraph around S : $\cup_{v \in S} \{u \mid d(u, v) \leq h\}$
S	Edge set for link prediction	$d(u, v)$	Shortest path distance between nodes u and v
$\tilde{A}$	Row-normalized adjacency matrix	D	Degree matrix
$\hat{A}$	Adjacency matrix with self-loops	I	Identity matrix
$\top$	Transpose operation	F	Set of hyperedges
T_E	Set of edge types	T_V	Set of node types
$\mathcal{H}$	Hypergraph, i.e., $\mathcal{H} = (V, F)$	F_i	A hyperedge $\in F$, which is a set of nodes
g	Prediction function	$MLP^{(l)}$	l th multi-layer perceptron (MLP)

EDA flow from HLS to physical design. It shows how ML algorithms help address complex design challenges by optimizing toolchain selection, predicting performance outcomes, and automating **design space exploration (DSE)**. Early ML techniques, like **support vector machine (SVM)** [40] and **K-nearest neighbor (KNN)** [62], required manual circuit encoding and feature extraction, while models like **convolutional neural network (CNN)** [60] and **recurrent neural networks (RNN)** [191] were not well-suited for graph-structure circuit data. GNNs, however, automatically learn task-specific embeddings directly from circuit graphs, enabling more effective and scalable solutions.

2.4.2 Graph Types.

We start by presenting the definition of a graph as follows.

Definition 1. $\mathcal{G} = (V, E)$ denotes a graph consisting of a set V of nodes and a set $E \subseteq V \times V$ of edges. The node feature matrix $X \in \mathbb{R}^{n \times k}$ has $n = |V|$ rows corresponding to nodes in $\mathcal{G}$ and k is the dimensionality of each node's feature vector, e.g., for a node v , $k = |x_v|$. The adjacency matrix $A \in \{0, 1\}^{n \times n}$ encodes the edge structure of $\mathcal{G}$, with $A\{u, v\} = 1$ if and only if $(u, v) \in E$.

Definition 2. A directed graph (or digraph) is a graph $\mathcal{G} = (V, E)$, where each edge $(u, v) \in E$ indicates a directed connection from node u to node v .

Definition 3. An undirected graph is a graph $\mathcal{G} = (V, E)$ where V is a set of nodes and $E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$ is a set of unordered pairs. That is, an edge $\{u, v\} \in E$ implies a bidirectional connection between u and v . The adjacency matrix $A \in \{0, 1\}^{n \times n}$ is symmetric: $A = A^T$.

Definition 4. A hypergraph is a generalization of a graph defined as $\mathcal{H} = (V, F)$, where V is a set of nodes and F is a set of hyperedges. Each hyperedge $F_i \in F$ is a non-empty subset of nodes: $F_i \subseteq V$, with $|F_i| \geq 2$. Unlike standard graphs, a hyperedge can connect more than two nodes simultaneously.

Definition 5. A heterogeneous graph is a graph $\mathcal{G} = (V, E, \tau_V, \tau_E)$ where

$$\tau_V : V \rightarrow T_V, \quad \tau_E : E \rightarrow T_E,$$

are type-mapping functions assigning each node and edge a type from the finite sets of node types T_V and edge types T_E , respectively, with $|T_V| > 1$ or $|T_E| > 1$. In contrast, a homogeneous graph is a graph where all nodes and edges have the same type, i.e.,

$$|T_V| = 1 \quad \text{and} \quad |T_E| = 1.$$

2.4.3 Generic GNN Architecture.

Researchers have proposed a variety of GNN architectures, including the **graph convolutional network (GCN)** [96], **GraphSAGE** [70], **deep graph convolutional neural network (DGCNN)** [225], **graph isomorphism network (GIN)** [208], **message passing neural network (MPNN)** [66], **graph attention network (GAT)** [186], and **principal neighborhood aggregation (PNA)** [54], among others. Next, we discuss the fundamental operations underlying any GNN architecture (e.g., aggregation, readout, and pooling) that are relevant for downstream tasks such as classification, regression, and clustering. We also provide examples from key GNN architectures such as GraphSAGE [70], GCN [96], GIN [208], and PNA [54].

Definition 6. **GNNs** produce a representation (i.e., *embedding*) Z by learning on structure and node features of $\mathcal{G}$, which supports graph-based ML tasks. More specifically, a GNN accepts a graph $\mathcal{G}$ as input and produces an embedding z_v for each node v within V .

Aggregation Functions: These node embeddings are updated across multiple iterations through a process known as *neighborhood aggregation*, also referred to as *message passing*, where each node updates its embedding by aggregating its own features with those of its neighbors. Recall that A is the adjacency matrix, which is essential in this operation to extract the neighbors of each node. The aggregation function is typically order-invariant, combining the features of a node and its neighbors using operations such as summation (adding all features), maximum (taking the element-wise max), or averaging. The use of a trainable weight matrix controls the aggregation by modulating the relative importance of the node's own features versus those of its neighbors.

$$Z^{(l)} = \text{Aggregate} \left(A, Z^{(l-1)}; W^{(l)} \right). \quad (1)$$

Here, $Z^{(l)}$ denotes the node embedding matrix at the l th iteration (or message passing layer), while $W^{(l)}$ is a learnable weight matrix. The initial node features X are represented by $Z^{(0)}$. The final node embeddings are obtained at layer L , i.e., $Z^{(L)}$, which corresponds to the total number of message passing layers. Note that this is a generic, abstract equation, and different GNN architectures employ different aggregation functions tailored to their design.

For example, one common variant of GraphSAGE [70] uses the mean aggregator, where each node aggregates the average of its neighbors' features and concatenates it with its own embedding. The process can be expressed in matrix form as:

$$Z^{(l)} = \sigma\left(W^{(l)} \cdot \text{concat}\left(Z^{(l-1)}, \tilde{A} \cdot Z^{(l-1)}\right)\right). \quad (2)$$

Here, σ is a non-linear activation function such as ReLU. The matrix $\tilde{A}$ is a normalized version of the adjacency matrix A , typically defined as $\tilde{A} = D^{-1}A$, where D is the degree matrix with $D_{v,v} = \sum_u A_{v,u}$. This normalization ensures that each row of $\tilde{A}$ represents an average over a node's neighbors. The operation $\text{concat}(\cdot, \cdot)$ denotes row-wise concatenation. The neighborhood of a node v is defined as $\mathcal{N}(v) = \{u \in V \mid (u, v) \in E\}$, and the final node embeddings are obtained after L such message passing layers.

GCNs [96] update all node embeddings simultaneously using a normalized adjacency matrix that includes self-loops. The matrix-form update is:

$$Z^{(l)} = \sigma\left(\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}} Z^{(l-1)} W^{(l)}\right), \quad (3)$$

where $\hat{A} = A + I$ is the adjacency matrix with self-loops, I is the identity matrix, and $\hat{D}$ is its corresponding degree matrix with $\hat{D}_{v,v} = \sum_u \hat{A}_{v,u}$. This symmetric normalization ensures stable scaling of node features across layers, and σ denotes a non-linear activation function.

GINs [208] use an aggregation function implemented via a MLP to capture graph structure more effectively. The matrix-form update is:

$$Z^{(l)} = \text{MLP}^{(l)}\left((I + A)Z^{(l-1)}\right). \quad (4)$$

The function $\text{MLP}^{(l)}$ is applied row-wise to the resulting matrix.

PNA [54] enhances message passing by combining multiple aggregation functions with degree-based scaling. The matrix-form update is:

$$Z^{(l)} = \text{MLP}^{(l)}\left(\text{concat}\left(\bigcup_{i=1}^{AF} \bigcup_{j=1}^{SF} \text{Scale}_j(D) \cdot \text{Aggregate}_i(A, Z^{(l-1)})\right)\right). \quad (5)$$

The parameter AF denotes the number of aggregation functions, such as mean, max, min, and std while SF denotes the number of degree-based scaling functions, like identity, amplification, and attenuation. The function $\text{Aggregate}_i(A, Z^{(l-1)})$ applies the i th aggregator over neighbors, and $\text{Scale}_j(D)$ applies the j th scaling function elementwise to D . The concatenation collects these diverse scaled features before the $\text{MLP}^{(l)}$ is applied.

2.4.4 GNN Tasks.

The **readout function** aggregates the node embeddings produced after L layers of message passing into embeddings suitable for downstream tasks. For **node-level** tasks, the readout returns the final embedding of each node $v \in V$, z_v (e.g., from $Z^{(L)}$), while for **graph-level** tasks, a global

pooling function such as sum, mean, or max is applied over all node embeddings to produce a single graph embedding, i.e., z_G .

Different GNN architectures employ distinct readout strategies. GCN and GraphSAGE commonly use the node embeddings from the final layer $Z^{(L)}$ directly. In contrast, GIN improves representational power by aggregating embeddings from all layers $\{Z^{(0)}, \dots, Z^{(L)}\}$ through concatenation or summation, capturing multi-scale structural information [208].

Depending on the application, these embeddings are used in different learning settings:

Classification. The goal is to assign a discrete label to a node (or graph).⁴ The prediction function g is typically implemented as a softmax classifier:

$$\hat{y}_v = \text{softmax}(Wz_v + b), \quad \hat{y}_v \in \mathbb{R}^c, \quad (6)$$

where c is the number of classes, and W, b are trainable parameters.

Regression. The model predicts a continuous value rather than a class label. The output is typically a linear transformation of the embedding:

$$\hat{y}_v = Wz_v + b, \quad \hat{y}_v \in \mathbb{R}. \quad (7)$$

Clustering. Node embeddings z_v are used directly for similarity-based grouping using algorithms such as K-Means. No explicit prediction function g is used; rather, the clustering is applied on the learned embedding space.

These task-specific functions operate on either node embeddings z_v or graph-level embeddings z_G , depending on whether the task is node-level or graph-level.

Link Prediction. In GNN-based link prediction, the goal is to predict the existence of links between nodes in the graph. This is done in [224] by extracting an enclosing subgraph around each target link. Given a graph $\mathcal{G} = (V, E)$ and a set of target links $S \subseteq V \times V$, the subgraph $\mathcal{G}_{(S,h)}$ is derived from $\mathcal{G}$ by considering all nodes within a certain distance h from each node in S :

$$\mathcal{G}_{(S,h)} = \cup_{v \in S} \{u \mid d(u, v) \leq h\}, \quad (8)$$

where $d(u, v)$ is the shortest path distance between nodes u and v , representing the local neighborhood around the target link. These subgraphs encapsulate the network context around the links, enabling more accurate prediction of missing edges. Once the subgraphs are formed, standard graph classification techniques can be applied. The labels assigned to the target links correspond to the labels of the respective subgraphs.

2.4.5 Loss Functions.

Loss functions are central to optimizing GNNs, as they quantify the gap between predictions $\hat{y}_v$ and ground truth y_v . Their design varies with task objectives, data properties, and output constraints [206]. In **supervised** learning, where models are trained on labeled data with known target outputs, loss functions directly measure prediction accuracy against ground truth labels. Classification tasks typically employ cross-entropy or **negative log-likelihood (NLL)** loss, often with weighting schemes to handle class imbalance. For regression tasks, **mean squared error (MSE)**:

$$\text{MSE} = \frac{1}{|V|} \sum_{v \in V} (\hat{y}_v - y_v)^2, \quad (9)$$

mean absolute error (MAE):

$$\text{MAE} = \frac{1}{|V|} \sum_{v \in V} |\hat{y}_v - y_v|, \quad (10)$$

⁴The prediction function g can also be tailored to perform edge or link prediction tasks, where the target is a pair of nodes (i.e., a potential edge) rather than individual nodes or graphs.

and **mean absolute percentage Error (MAPE)**:

$$\text{MAPE} = \frac{1}{|V|} \sum_{v \in V} \left| \frac{y_v - \hat{y}_v}{y_v} \right|, \quad (11)$$

are widely used, where MSE emphasizes large errors and MAPE offers scale-invariant evaluation. To assess fit quality, the coefficient of determination R^2 :

$$R^2 = 1 - \frac{\sum_{v \in V} (y_v - \hat{y}_v)^2}{\sum_{v \in V} (y_v - \bar{y})^2}, \quad (12)$$

which measures the variance explained by predictions, while the Pearson correlation coefficient r :

$$r = \frac{\sum_{v \in V} (\hat{y}_v - \bar{\hat{y}})(y_v - \bar{y})}{\sqrt{\sum_{v \in V} (\hat{y}_v - \bar{\hat{y}})^2} \sqrt{\sum_{v \in V} (y_v - \bar{y})^2}}, \quad (13)$$

which captures the linear association between predictions and ground-truth, where $\bar{y}$ and $\bar{\hat{y}}$ denote their respective means. In contrast, **unsupervised** learning operates without explicit labels, requiring loss functions that capture meaningful structural patterns in the data itself. For example, contrastive and cosine similarity losses are widely used to learn meaningful graph embeddings [219]. Regularization methods such as L_1/L_2 losses and label smoothing further enhance model generalization and stability.

2.4.6 Model Performance Evaluation.

Classification tasks employ accuracy

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (14)$$

for balanced data, F1-score

$$\text{F1-score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (15)$$

for imbalanced scenarios, and AUC-ROC for class separability, where $\text{Precision} = \frac{TP}{TP+FP}$ and $\text{Recall} = \frac{TP}{TP+FN}$ ⁵. F1-Macro/micro for multi-class problems and edge accuracy plus average precision for edge-level tasks [206]. Regression tasks utilize MAE for error quantification, MAPE for scale-normalized assessment, R^2 for variance explanation, and r for linear correlation. Link prediction is typically judged by AUC, which measures how well the model ranks true edges above non-edges [224].

3 Proposed Taxonomy

In this section, we present our proposed taxonomy, starting with an overview of the general pipeline for applying GNNs to IC-related problems, as illustrated in Figure 4. The process begins by representing a circuit—whether digital or analog and provided in GLN, RTL, or transistor-level format—as a graph. The graph’s structure, which may be directed, undirected, hypergraph, heterogeneous, or homogeneous, depends on the specific task. Various node and/or edge features are extracted from the circuit based on the target objective. The graph is then input into the GNN model to perform ML tasks such as node/graph classification, regression, clustering, and more. Aside from the various GNN architectures outlined above, researchers can design custom GNN

⁵ TP = True Positives (correctly predicted positive samples), TN = True Negatives (correctly predicted negative samples), FP = False Positives (negative samples incorrectly predicted as positive), and FN = False Negatives (positive samples incorrectly predicted as negative).

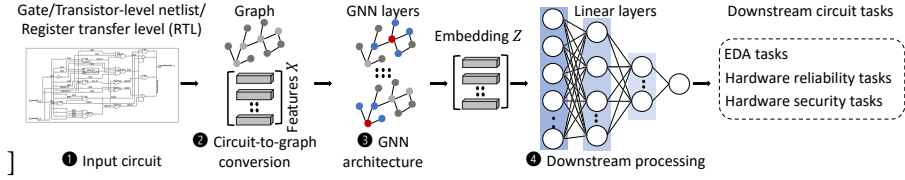


Fig. 4. General pipeline—the employment of GNNs toward addressing different IC-related tasks.

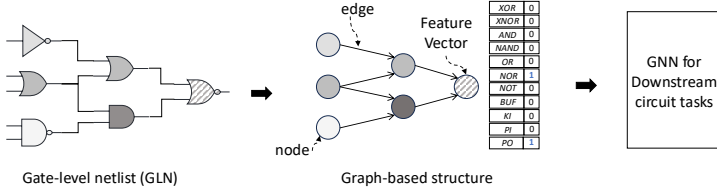


Fig. 5. An example of GLN to directed graph conversion.

architectures tailored to specific IC-related tasks. GNN models can also be combined with other ML techniques, such as **long-short-term memory (LSTM)** networks [74] and **reinforcement learning (RL)** [86] to enhance their capabilities.

Consider Figure 5, for example. Here, the GLN is transformed into a directed graph structure (other graph types are also applicable, but we show one example here), where each Boolean gate in the GLN is represented as a node, and the connections between gates are modeled as edges. The gate type and other circuit properties associated with each node can be used to create a feature vector. An example feature vector, shown in the figure, indicates that the gate type is a NOR gate, which is connected to the primary output of the circuit. The extracted node or edge features within the graph can then be leveraged for further GNN tasks to address IC-related problems.

The following sections provide an in-depth review of the IC-related areas outlined in our taxonomy: EDA, reliability, and security. For each stage in the IC design flow, we categorize and discuss the related works based on the corresponding GNN task, e.g., node/edge classification, regression, and integration with RL. We follow this categorization throughout this section.

3.1 GNNs for EDA

Table 5 provides a summary of selected works employing GNNs for various EDA-related tasks across the circuit design flow, which we will discuss in the following sections. It further categorizes GNN-based EDA platforms by their corresponding tasks, highlighting how different GNN tasks are employed and the specific EDA problems they address. As a complementary analysis, Table 8 systematically organizes the feature elements used in GNN-based EDA applications.

3.1.1 Behavioral and Logic Design.

Node/edge classification. D-SAGE [184] investigates the mapping of operations in HLS for **field programmable gate arrays (FPGAs)**. The authors observe that the structural context surrounding arithmetic operations affects how these operations are allocated to FPGA resources, thereby influencing timing performance. Their framework leverages these mapping behaviors to characterize the timing of arithmetic modules, while accounting for mapping considerations.

Edge regression. Lopera *et al.* [122] represent RTL design components as DAGs with multi-dimensional features on both nodes and edges to estimate timing (i.e., delay and slew value) at the RTL level. They utilize two GNN architectures, MPNN and GCN, to extract node embeddings,

process them through an MLP, and perform edge-level regression tasks. However, this edge-level prediction approach does not directly model the design's critical path through GNN-based circuit models, which we anticipate would be more powerful in later Section 3.2.

Graph regression. Ferretti *et al.* [61] introduce GNN4HLS, a framework addressing HLS-driven DSE that reduces the synthesis efforts, through exploration of Pareto optimal implementations. They represent HLS designs using a hybrid **control and dataflow graph (CDFG)**, and train a GNN model to estimate the performance and cost of each implementation in the target function configuration space before selecting which configurations to synthesize. In contrast to [202, 203], this DSE-based approach explores the entire Pareto frontier of solutions rather than focusing solely on a single performance metric like latency.

GNN-RL. Wu *et al.* [202, 203] propose their framework to address the HLS DSE problem, with the consideration of user-defined constraints and/or Pareto optimization methods. Three modules are involved in the framework: a GNN-based predictor for assessing performance and resource utilization; an RL-driven, multi-objective DSE engine; and a code transformer that aids the other modules by extracting **data-flow graphs (DFGs)** from C/C++ source code and generating RTL code.

Representation learning. Li *et al.* [111] propose DeepGate to represent circuits in a singular and universally applicable format, aiming to mitigate the reliance on diverse technology libraries and formats in IC design. They represent circuits as **and-inverter graphs (AIGs)**, and run logic simulations in a supervised fashion to obtain the signal probability on every node. A GNN model is then trained to accurately estimate the signal probability of every node.

Yang *et al.* [210] introduce a unified circuit heterogeneous graph representation for GLNs, Circuit Graph, which integrates topological and geometrical information in circuits with a linear time consumption to the scale of design. An accompanying GNN design, Circuit GNN, is also introduced to process these circuit graphs. In Circuit Graphs, there are two types of vertices, namely cells and nets. Pins connect the cells and nets and represent the topological information by topo-edges, and the cells that are geometrically close are linked by geom-edges to represent the geometry. Circuit GNN updates cells and nets representations by applying message passing to topo-edges and geom-edges separately and later fusing the messages.

Khan *et al.* [93] introduce DeepSeq, a method for learning representations of sequential circuits that captures their temporal correlations and dynamic behaviors. Traditional models like DAG-ConvGNNs [226] and DAG-RecGNNs [27] fail to handle dynamic temporal dependencies and cycles introduced by memory elements such as flip-flops. DeepGate and other models [195] are limited to combinational circuits. DeepSeq utilizes a specialized DAG-GNN architecture with a dual attention aggregation mechanism to effectively learn logic and transition probabilities across circuit nodes.

3.1.2 Logic Synthesis.

Node regression. Ghose *et al.* [65] focus on routing congestion prediction during synthesis. Their method can handle cross-graph alignment and generalize across unseen netlist graphs. By demonstrating poor performance results with the popular embedding methods Node2vec [67], Line [182], and DeepWalk [145], they find that matrix factorization along with clustering shows superior results. Moreover, they also observed the shallow, wide version of GraphSAGE performs better than the deep, wide GATs.

Kirby *et al.* [97] leverage GATs to anticipate routing congestion before the placement stage. They model circuits as undirected graphs, where gates are depicted as nodes and wires as edges, with initial features including cell type, function, and geometry. This research represents an early application of GNNs for circuit analysis.

Graph regression. De *et al.* [56] tackle inaccurate delay estimates in HLS tools for **application-specific IC (ASIC)** designs as compared to logic synthesis tools. They focus on GLNs and utilize a GNN to improve HLS delay prediction. The GNN model features a front-end graph MPNN to generate local structural features and a back-end for nongraph-based regression stages for delay prediction. The GNN's end task is to automatically learn to map downstream optimizations onto HLS critical paths with the ultimate goal of accurate delay prediction. They demonstrate that hybrid models incorporating both local (i.e., graph-based) and global (i.e., nongraph-based) features enhance prediction performance.

BoolGebra [113] is a graph learning approach for Boolean algebraic manipulation to improve fundamental logic synthesis. It takes initial feature embeddings from both structural and functional information of AIGs. The method samples designs, prunes the less effective ones, and evaluates the top results to determine their effectiveness in optimizing the AIG nodes. BoolGebra integrates multiple optimization techniques such as rewriting, resubstitution, and refactoring within a single AIG traversal, using static and dynamic feature embeddings for each node in the AIG to evaluate and optimize each node's potential. It reduces the search space and efficiently locates optimized solutions, showing generalization from small to complex designs.

GNN-RL. Wu *et al.* [200] introduce a hybrid framework that predicts quality-of-result by jointly considering circuits' structure and temporal information, such as the sequence of synthesis steps. To exploit spatio-temporal information, they propose two hybrid GNN-based models: the first utilizes a GNN to characterize designs through "supernodes" that encode temporal information, and the second combines a GNN for spatial learning with an LSTM for temporal learning.

Imitation learning. Zhou *et al.* [231] introduce a heterogeneous, directed GNN model for encoding timing graphs, using imitation learning for training. The model is designed to accelerate gate sizing based on Lagrangian relaxation [63]. The approach demonstrates strong transferability to larger, previously unseen graphs.

3.1.3 Partitioning, Floorplanning.

Node clustering. Lu *et al.* [125] focus on the partitioning of modules in 3D ICs, specifically targeting the modules to layers assignments within a 3D stack. To mitigate the overhead of routing associated with traditional bin-based partitioning, they initially introduce a hierarchy-aware edge-contraction algorithm. Subsequently, they use an unsupervised GNN framework to solve the layer partitioning and assignment as a weighted k-means clustering problem. Besides hierarchy information, the framework incorporates timing parameters as feature vectors.

Graph generation. Liu *et al.* [120] introduce GraphPlanner, a variational GCN-based model designed for chip floorplanning. This model optimizes the mapping between circuit connectivity and physical wirelength given an area constraint. GraphPlanner not only determines the physical locations of macros and standard cells but also addresses the complex NP-hard problem of chip floorplanning by generating a synthetic dataset for training. Inspired by Google AlphaGo [137], this is achieved through a holistic approach rather than the sequential placement methods used in traditional models. The synthetic dataset helps the model understand a wide range of statistical connectivity distributions, aiding in generalizing to real-world scenarios. The model uses c-spectral clustering to merge objects connected by heavy hyperedges into clusters, informing the placement process guided by the learned mappings.

Graph clustering. Hong *et al.* [76] propose GraphClusNet, a hierarchical GNN that performs circuit graph clustering in a multistage process. The authors emphasize the importance of extracting relevant functional blocks from a recovered netlist as a crucial step in hardware assurance for manufactured or programmed ICs. GraphClusNet addresses the challenge of identifying such blocks without known ground truth partitions by using an unsupervised learning approach. The

strategy involves dividing the circuit into two main groups: the central functional blocks (like a processor core) and the peripheral circuits, which aids in design verification. The clustering task involves finding minimal connections between these two groups, using a normalized-cut objective to minimize the connections cut during clustering.

GNN-RL. Mirhoseini *et al.* [137] utilize RL to approach floorplanning in an industrial setting, employing an edge-based GCN for modeling the high-level netlist of modules. On a practical note, this work has received significant concerns from the wider community [134].

DeepTH [229], an RL-based three-head policy gradient placer, is proposed to generate chip floorplans from scratch without the need for pretraining. The architecture of RL networks generally only features two heads, one to estimate the policy and value. In this study, however, a third head, the reconstruction head, is proposed to recover the current placement's visual representation, allowing for visual reconstruction and extracting more reliable features. The authors denote the residual sum of squares between the real canvas and the reconstructed canvas as the reconstruction error and adopt it as an exploration bonus. The final reward function for the RL model would be the sum of the half-perimeter wire length and the exploration bonus, which encourages explorations while placing chip objects and then alleviates the sparse reward problem effectively.

3.1.4 Placement and Routing.

Node clustering. Lu *et al.* [129, 130] propose a framework for generating cell clusters to guide placement. It utilizes the GraphSAGE model to learn node embeddings from initial features like hierarchy information and memory affinity, then employs weighted k-means clustering.

Agnesina *et al.* [6] propose a GraphSAGE-RL based framework for tuning placement by utilizing the embedded features from the GNN model. They extract the initial features from the netlist graph, such as area, fanout, and gate type, to process through GraphSAGE. Additionally, global descriptions of circuits are taken as features to incorporate into the overall RL framework. A similar approach is presented in [5], where the authors also address partitioning in 3D ICs.

Node classification/regression, edge regression. Guo *et al.* [68] focus on predicting arrival times and slacks to enable timing-driven placement without relying on **static timing analysis (STA)**. They represent circuits as heterogeneous graphs, with edges for nets and cells, and nodes for cell pins. The features used are inspired by timing engines, including pin capacitances and net distances. The authors note that excessively deep GCN architectures, or "brute-force stacks," do not yield strong performance, emphasizing the need for careful exploration of GNN architectures.

Chen *et al.* [48] address challenges of routability estimation in modern VLSI circuits by developing a GNN model to predict short violation. By constructing an adjacency matrix to represent the connectivity between different tiles in the circuit layout, they use GraphSAGE to effectively map local tile features (e.g., cell density, relative position). The model demonstrates improved binary classification quality and inductive learning capabilities compared to traditional (i.e., non-GNN) ML frameworks including NN [180], U-Net [31], and J-Net [115].

Edge regression. Xie *et al.* [207] propose a GAT-based framework designed to estimate net lengths before placement. This framework utilizes partitioning-related features, such as cuts and cluster assignments, which capture nodes—nets relationships by serving them as directional edge features. The model also incorporates hypergraphs that include cell nodes.

Graph regression. Baek *et al.* [31] introduce a method for **design rule check (DRC)** during placement, accounting for both pin access and routing congestion. The approach begins with creating a pin proximity graph and modeling spatial relationships between cell pins and pin-to-pin conflicts. They then utilize a GNN model to capture local pin accessibility from the pin proximity graph, and they use U-Net to represent local and global routing congestion by grid-based features.

Lee *et al.* [109] present a framework for predicting the placement of the locations and aspect ratios for design blocks with flexible rectangles while minimizing the cost of wirelength. The

features are characterized as either static or dynamic. The GNN encodes these features and passes through five deconvolutional layers to map the vector to a three-dimensional tensor, representing the location as a grid and the aspect ratio from a set of aspect ratios.

GNN-RL. Cheng *et al.* [51] propose a framework that integrates RL with gradient-based optimization to address both placement and routing tasks. They employ a multi-view embedding model to capture global graph-level and local node-level information. Moreover, they incorporate distillation techniques to enhance the exploration of the search space.

Chen *et al.* [42] introduce an RL-guided framework to solve the detailed routing problem in custom circuit designs. This framework utilizes a set of placed cells, nets, and specific routing constraints to generate a DRC solution that minimizes overall routing costs while adhering to layout robustness constraints like symmetry and topology-matching. The authors represent routing configuration as a heterogeneous graph with vertices representing segments, paths, constraints, and edges categorized by their connection types. They then employ GraphSAGE layers to process these graphs and produce embeddings representing graph features, which inform the routing optimization process. The framework effectively learns optimal routing strategies by iteratively refining solutions and updating parameters to minimize loss.

3.1.5 Timing Closure.

Graph regression. Lu *et al.* [127] aim to predict post-route TNS to identify the potential of failing timing closure during design runs. To achieve this, they extract features from earlier stages, like placement and clock delivery, and represent their embeddings through GNNs. Additionally, the sequential modeling of the design flow is complemented by LSTM networks.

Ye *et al.* [217] introduce a deep EdgeGAT model to predict post-layout **path-based analysis (PBA)** of timing results using costly **graph-based analysis (GBA)** results. The model utilizes multiple EdgeGAT layers for processing multi-dimensional node/cell and edge/net features to predict cell slew and delay. This work ultimately showcases the potential of EdgeGAT to learn path information and achieve accurate timing predictions. The results demonstrate that predictions of PBA timing results are accurate and reduce timing pessimism of GBA while achieving an average 24.80x speedup over PBA using the golden commercial STA tool PrimeTime [179].

GNN-RL. Lu *et al.* [128] introduce an RL framework combined with a GNN for gate sizing. The authors highlight that subgraph extraction within the GNN is critical for accurately predicting the **total negative slack (TNS)** impact of sub-circuits on the overall circuit performance.

Representation learning. SyncTREE [78] presents a specialized tree-based GNN architecture aimed at enhancing timing analysis in IC design, particularly for RC-trees. The architecture includes a two-pass message-passing mechanism for graph embedding, utilizing a bottom-up and top-down approach, along with a tree contrastive loss to guide learning. The GNN employs a closed-formula-based method for rapid timing analysis. The end task of the GNN is to predict timing metrics such as delays and slews accurately and efficiently. The experiments conducted in the study evaluate the performance of SyncTREE against industry-standard numerical analyses on real IC design data and validate the effectiveness of the SyncTREE model in enhancing timing analysis for IC interconnects.

3.1.6 Verification, Testing.

Node classification. Ma *et al.* [132] present a multi-stage GCN-based framework to predict observation points within a netlist. They label nodes as difficult or easy to observe and then train a classifier to identify optimal locations for inserting observation points. This approach aims to maximize fault coverage while minimizing the number of observation points and test patterns.

Hung *et al.* [82] propose a diagnosis framework to improve yield-learning in **monolithic 3D (M3D)** ICs by addressing systematic defects that degrades power and performance due to immature fabrication. The authors introduce involves the use of two GNN models to locate faults at the

tier level and in **monolithic inter-tier vias (MIVs)**. The framework converts the circuit under diagnosis into a two level heterogeneous graph, with fault sites and MIVs represented as nodes connected by edges (input-pin-to-output-pin and net-stem-to-net-branch connections), which helps in fault localization. The authors also introduce a GNN-based reordering and pruning algorithm that leverages the fault predictions and precision-recall curves to improve the final **automatic test pattern generation (ATPG)** diagnosis reports.

Wang *et al.* [192] introduce the GRAND framework aimed at enhancing the diagnostic resolution of ICs during initial bring-up and production excursions. The framework utilizes GNNs to analyze circuit topology, which allows it to more effectively identify true faults among the diagnostic callouts. By incorporating circuit topology and structural information, which were previously underutilized, GRAND exploits the interdependencies among fault candidates, enriching the initial feature set. Through experiments on various benchmark circuits, including both proprietary and open source designs, GRAND demonstrates the ability of the proposed method to improve diagnostic resolution by identifying true fault candidates without incurring substantial computation overhead, which suggests its feasibility for integration into industrial post-silicon debug workflows.

Node regression. Zhang *et al.* [227] introduce a supervised learning method for estimating toggle rates to assess power consumption without relying on gate-level simulations. They represent netlists as graphs, utilizing register states and inputs from RTL simulations as features, while using the toggle rates of combinational gates as labels.

Zhu *et al.* [232] proposes RC-GNN, a customized GNN model to estimate interconnect delays in RC circuits during the signoff stage. The model treats the RC network as a heterogeneous graph with nodes representing capacitances and edges representing resistances, differentiated by positive, negative, and neutral types. The RC-GNN architecture features a custom GNN architecture that performs four distinct message passing functions: negative passing, neutrality passing, positive passing, and feature aggregate. Through these specific node message updating and reducing functions, the aggregated nodes are processed through a five-layer MLP to predict net delays. The model addresses over-smoothing issues and computational costs by restricting the depth of the message propagation scope and the graph traversal order at each iteration.

GNN-RL. Shi *et al.* [169] present an RL agent that combines a GNN with a deep Q-learning network to optimize test coverage. Specifically, they model circuits as directed graphs and design a graph-based value network to evaluate the action values for placing various test points.

3.1.7 Analog, Mixed Signal, and Transistor Design.

Node classification. Kunal *et al.* [106] automate analog design by leveraging GNNs to identify hierarchical functional blocks. They represent the circuit netlist as an undirected bipartite graph and use GNNs for approximate subgraph isomorphism to classify circuit elements into different classes based on their subcircuit affiliation. The problem is formulated as a supervised node classification, employing a GNN to propagate and extract features from graph structure. The model is trained to aggregate characteristics of neighboring nodes, enabling it to identify basic and complex structures within the circuit, such as primitives and sub-blocks like amplifiers and oscillators. This method aims to streamline tasks such as layout optimization and hierarchical design generation.

Graph regression. Zhang *et al.* [223] apply GNNs to the design of distributed analog and mixed-signal (AMS) circuits, particularly operating at high frequencies, such as 5G and 6G electronics, where wavelengths are comparable to or smaller than circuit components. Their GNN model proves effective in both simulating these circuits and automating the design process for complex and expert-driven ones. The approach significantly reduces simulation times and leads to the discovery of new structures and templates through GNN-based design.

Ren *et al.* [156] aim to enhance the efficiency and accuracy of pre-layout simulations for AMS circuits by predicting layout parasitics and device parameters. They utilize ensemble modeling (i.e., GraphSage, Relation GCN, and GAT) to process the circuit graphs, which improves model accuracy across a diverse set of prediction values. Building on this work, Liu *et al.* [117] incorporate dropout as an effective method for uncertain prediction during the Bayesian optimization for transistor sizing automation. They contend that parasitic prediction within the optimization loop ensures compliance with all design constraints. In contrast, optimization based solely on schematics often fails to meet several constraints when verified with parasitic estimations.

Representation learning. Chen *et al.* [43] focus on symmetry matching in AMS circuits by introducing an unsupervised, inductive GNN framework, that is, both generalizable and capable of extracting symmetry constraints at system and device levels. They propose a heterogeneous multi-graph representation for modeling interconnections, along with a circuit feature embedding algorithm that effectively represents circuits by capturing their most representative substructures.

Dong *et al.* [57] introduce CktGNN, an analog IC framework that encodes circuit graphs using a two-level nested GNN to generate optimized circuit topologies and device sizes. In the traditional nested GNN framework, the node representation encodes a rooted subtree around each node. However, this method of encoding rooted subtrees leads to an expressive power of GNNs, that is, upper bounded by the one-WL test [208]. To increase the graph representation ability, it first represents the input DAG of the circuit as a combination of subgraphs with a pre-designed subgraph basis and then leans representations of subgraphs with inner GNNs. The outer GNN then performs directed message passing using the learned node embeddings to learn a representation for the whole graph.

Shahane *et al.* [167] introduce the GCX framework that enhances analog IC design automation by leveraging graph structure learning alongside GNNs. This framework operates within a semi-supervised learning framework to reduce reliance on large volumes of labeled data. The approach begins by constructing a detailed feature vector from the geometric representation of circuits, enriched with technology-specific information. This forms the basis for a graph-based model that integrates both few-shot and zero-shot learning to improve prediction for new, unseen circuits. Furthermore, GCX includes two novel optimization algorithms to optimize sample selection to achieve optimal circuit configurations that meet design requirements.

3.1.8 Circuit Design Completion.

Graph classification. Said *et al.* [163] addresses the challenge of completing partially designed circuits by identifying missing components and their connections using GNNs. A GNN architecture called the **feature-enhanced graph isomorphism network (FEGIN)** is proposed to combine the representation strengths of GNNs with the static graph descriptor's ability to capture rich topological information. The framework is divided into two tasks: a graph classification task to identify the missing components and a link prediction task to find the optimal placement of the predicted component to all the other components in the given circuit. They also develop and introduce three new circuit datasets for testing and implementation. The effectiveness of FEGIN against fifteen different graph learning models, including nine GNNs and six graph descriptors, demonstrates its efficacy in automating complex circuit design tasks.

3.2 GNNs for Hardware Reliability

Most recently, GNNs have been used to tackle reliability problems. We review selected works below and summarize them in Table 6. This table also organizes the reviewed platforms by their GNN tasks, illustrating how different tasks are utilized to solve specific reliability problems. Additionally, Table 9 presents a detailed taxonomy of the feature elements employed across these works.

3.2.1 Aging Delay Prediction. Node regression. Chen *et al.* [46, 47] propose a GCN framework to predict aging-induced transistor degradation in analog ICs. Recognizing that analog circuits comprise various components such as transistors, capacitors, and resistors, the authors employ a heterogeneous graph representation and a GNN architecture to address this task.

Graph regression. Ye *et al.* [216] present a temporal-spatial GNN framework to predict circuit aging delay according to the requirements of different usage scenarios and circuits. The framework considers standard cells as homogeneous graphs, allowing a temporal network to capture dynamic stress biases and a spatial network to understand the circuit's structure. The combination of **gate tanh unit (GTU)** [185] and GraphSAGE shows superior performance aging delay prediction over other network candidates. Compared to the work [47] using static parameters for device features, this framework can make refined predictions under different usage scenarios.

Subgraph regression. GNN4REL [16] proposes a GNN-based framework to predict delay degradations of any specified timing path in digital ICs, considering factors like process variations and device aging. It enables designers to effectively size guardbands to ensure correct operation without needing STA or accessing sensitive transistor aging models during inference. The framework achieves this by extracting subgraphs around the timing paths of interest and performing subgraph regression.

3.2.2 Functional De-Rating Prediction.

Graph regression. Balakrishnan *et al.* [33] model GLNs using GCNs to predict **functional de-rating (FDR)** due to **single event upset (SEU)** fault in the circuit. They represent GLNs as probabilistic graphs and transform the graphs into a feature matrix using the Node2Vec [67] algorithm and adjacency matrix. The GCN is then trained to model the circuits' structural information and predict the FDR. Their further work [34] extracts the structural features from probabilistic graphs through GraphSAGE instead of Node2Vec. In this work, the embedded features combined with SEU fault injection simulation data are used to train a DNN to predict the FDR.

3.2.3 Environmental Variation.

Node classification. In [138], transistors that prolong a circuit's delay when degraded are identified for hardening using a GNN, specifically a GAT. A heterogeneous graph is used to capture the transistor-level circuit's topology and its impact on the analysis.

3.2.4 Fault Diagnosis and Evaluation.

Node classification. The work in [209] proposes a fault diagnosis method based on the combination of K-means and GCN for analog circuits. It uses K-means to cluster similar fault characteristics of the measured circuit, making it easier to analyze the signal data. After clustering, a graph is constructed based on the relationship of grouped signals. The GCN model is then trained to detect faults more accurately and efficiently than traditional or random GNN methods. This method has superior fault diagnosis performance in the study of through-silicon via defect modeling. Future work can focus on extending to different applications and improving the model's accuracy.

Graph regression. Koufopoulou *et al.* [103] introduces a methodology to evaluate hardware safety properties, specifically fault injection, of ICs generated through HLS, utilizing GNNs for efficient evaluations. This method represents GLNs as directed graphs and employs GNNs to perform graph regression tasks aimed at predicting hardware safety metrics, including critical and detection error rates. By introducing diversity in redundant HLS modules, the enriched dataset enables GNN to be trained to predict fault vulnerability metrics accurately. Future works could explore more advanced GNNs to improve prediction accuracy.

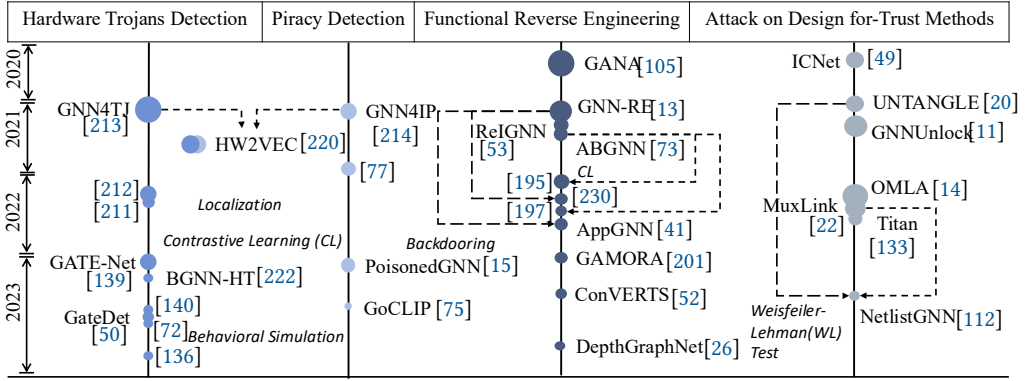


Fig. 6. GNN-based frameworks addressing various IC supply chain threats. Note that circle sizes are representative of the citation numbers.

3.3 GNNs for Hardware Security

GNNs have been used to address various hardware security concerns, like detection of HTs or IP piracy, functional RE, and attacks on DfTr methods, as shown in Figure 6. We review selected works and extract their key information in Table 7. This table organizes the security-focused GNN works by task, showing how various models are used to address specific security challenges. The comprehensive feature elements utilized across these works are categorized in Table 10.

3.3.1 HT Detection.

Node classification. Zhan *et al.* [222] introduce BGNN-HT model for HT detection at GLNs without a golden model or manual feature extraction. The input GLNs are first preprocessed into directed graphs. The BGNN-HT model uses a bidirectional GNN through two convolution channels to analyze cell features from both fan-in and fan-out perspectives, aggregating information and assessing the HT properties of cells based on their surrounding cells to predict whether a cell is part of an HT. Compared to GATE-Net [139] and the work in [212], it is not limited to the type of HTs and the scope of circuits.

Cheng *et al.* [50] introduce GateDet, a golden-free gate-level HT detection framework that leverages a **bidirectional graph convolutional network (BiGCN)** with a timely information fusion strategy. GateDet models netlists as directed graphs, preserving the structural information of the circuit. To handle large-scale design, it utilizes GraphSAINT to sample the graph, which reduces the training load and addresses the sample imbalance and structural incompleteness issues. The BiGCN then, with timely information fusion, expands the node aggregation scope and enables feature capture beyond that of traditional GNNs. The trained model outputs a list of gates likely to be a part of an HT, facilitating further manual review and removal. Future improvements could involve combining non-structural features, developing a more efficient directed graph framework, and creating a balanced circuit graph sampling scheme.

Meng *et al.* [136] introduce a layout-level HT detection algorithm that utilizes GNNs to identify and localize HTs without golden ICs. They use the GraphSAINT to detect HTs in ICs by analyzing the converted graph from the IC layout and capturing both the global structure of the layout and the local details of each cell and its neighbors. The GNN model flags cells with low-confidence classification, indicating potential suspicious HT regions. Then, the flagged cells are grouped into clusters to differentiate between HT-inserted and HT-free layouts, aiding in the localization of HT cells. Future work aims to improve the efficiency of finding all hidden HTs.

Pan *et al.* [140] present a method for HT-nodes detection in GLNs without a golden model. The method involves converting GLNs into directed graphs and feeding them into a unioned GNN, integrating information from input, output, and neighboring connections. Forward GNN learns the characteristics of gate-level information flow, and reverse GNN supplements node neighborhood information. The final node embeddings are generated by concatenating information from forward and reverse GNNs, enhancing the detection accuracy. Future work to improve accuracy and reduce architectural complexity still needs to be explored.

The usage of GNNs for detecting malicious circuit implementations that could lead to fault-injection attacks in cloud FPGA environments was also demonstrated by the work in [19].

Graph classification. Yasaei *et al.* [213] present the GNN4TJ framework to detect HT without prior knowledge of the design IP or the HT structure. It utilizes GNN to learn the design structures through extracted DFGs from RTL. GNN4TJ then performs graph classification to predict if HTs exist across the entire design. Other works extend this framework using GLNs [212, 220]. Furthermore, GNN-based platforms [211] have been developed to enable HT localization.

Hassan *et al.* [72] introduce a behavior-aware HT detection approach without golden IC reference. The approach first converts the GLN into an adjacency matrix and extracts essential features like node types, counts, and connectivity information through graph learning to model essential structural characteristics of the IC. This method then analyzes behavioral bit-flip patterns under abnormal conditions such as overclocking. The two-layer GCN model combines graph-level features with overclocking-based netlist simulation data to learn the distribution of HT-inserted features and achieve the end goal of classifying whether a given IC is HT-inserted or HT-free. However, data imbalance and varying HT block sizes may reduce prediction accuracy.

Representation learning. Muralidhar *et al.* [139] introduce GATE-Net to detect HTs by detecting triggers embedded in ICs without a golden model. The GATE-Net uses GCNs to process netlists as DAGs, representing each cell in the netlist as a node. GATE-Net then utilizes supervised **contrastive learning (CL)** to learn better representations, improving its ability to detect types of HTs, including combinatorial and sequential triggers. Unlike GNN4TJ [213], which requires costly RE for HTs injected after logic synthesis, GATE-Net works with GLNs to capture HTs injected at all design stages. However, its performance depends on the quality of the provided netlist and the known scope of cell types.

3.3.2 Piracy Detection.

Graph classification. PoisonedGNN [15] as a backdoor attack on GNNs to conceal HT and IP piracy. It designs and injects sub-circuit backdoor triggers automatically into RTLs or GLNs without affecting their functionality. The success of hiding HTs and IP piracy is evidenced by evaluations using GNN4TJ [213] and GNN4IP [214].

Hong *et al.* [77] propose a GNN-based framework for ASIC netlist recognition. They represent various adder circuits with different bit-widths using directed graphs, then embed node features and graph connections for graph classification. The GNN model is trained to recognize circuits with architectural differences. Further work could explore to apply GNN for complex circuit designs.

Hong *et al.* [75] introduce the GoCLIP framework, which employs GNN to identify targeted IP circuits from others by learning graph structure. The framework converts circuits into undirected graphs and utilizes GNN to extract feature vectors. It trains the GNN model to produce similar feature vectors for targeted circuits and different feature vectors for other circuits, facilitating easier differentiation. However, its effectiveness is influenced by sensitivity to structural changes and the complexity of embeddings.

Graph similarity. Yasaei *et al.* [214] present GNN4IP framework for detecting IP piracy. It identifies the theft by evaluating the similarity between the original circuit and a questionable

circuit. GNN4IP accepts different circuit types, like RTL or GLN. The method eliminates the need for watermarks or fingerprints, thereby reducing overhead and preventing removal attacks [8, 55]. The framework generates an embedding for each circuit from its underlying structure using the GNN, effectively serving as a “signature” of the design. The GNN then optimizes these embeddings to reflect the design similarities based on the distance of embedded vectors [70]. GNN4IP then uses the cosine similarity score between the embeddings to assess piracy between the two circuits. Yu *et al.* [220] introduce HW2VEC, a framework that combines GNN4TJ [213] and GNN4IP [214]. HW2VEC supports various graph types, including DFGs and **abstract syntax trees (ASTs)**.

3.3.3 Functional Reverse Engineering.

Node classification. Alrahis *et al.* [13] introduce the GNN-RE platform for functional RE. The approach extracts an undirected graph from a GLN. It represents an initialized feature vector for each node, including gate type with neighboring gates, input/output degree, and connectivity to primary ports. The platform can then identify which gates are components of specific sub-circuits or modules through node classification.

Bucher *et al.* [41] introduce AppGNN, exploring the GNN-RE [13] accommodation ability against approximate circuits. The trained GNN can generalize effectively under approximate computing scenarios by utilizing a node-sampling algorithm on exact circuits.

He *et al.* [73] introduce an **asynchronous bidirectional GNN (ABGNN)** framework for classifying sub-circuit in GLNs, specifically targeting arithmetic blocks like adders. To preserve the inherent circuit data flow, the ABGNN models GLNs as directed graphs. It trains two distinct GNNs to delineate the boundaries of adder blocks, separately aggregating information from fan-in and fan-out cones. The generated embeddings by both GNNs are then combined to form the final representation. Furthermore, other work [197] extends this framework to enhance the precision and reliability of datapath extraction within arithmetic blocks.

GAMORA [201] introduces a symbolic reasoning framework that leverages GNNs to RE functional blocks from GLNs. It takes GLNs in homogeneous AIGs and performs multi-task GNN to reason the boolean function of each node, allowing later functional block extractions on the annotated AIGs. The multi-task GNN performs boolean functional and structural aggregation with enhanced scalability, similar to conventional reasoning methods. It combines structural node embeddings with features to generate final node embeddings. Once the final node embedding is generated, the boolean function of each node will be identified through multi-task node classification. Unlike GNN-RE [13], the compressed features in node embeddings enable strong scalability, and multi-task GNN allows high-level extraction from the multi-labeled graph.

Zhao *et al.* [230] proposes a GNN-based functional RE method for circuit rewriting. The prediction accuracy can be enhanced by incorporating the employed features in the GNN-RE [13] with a set of functionality-aware features, which are derived from truth table information, signal probabilities, and **negation-permutation-negation (NPN)** class.

Chowdhury *et al.* [53] introduce ReIGNN, where GLNs are represented as graphs processed by GNNs to distinguish between state and data registers.

Kunal *et al.* [105] present GANA to RE analog circuits through a GCN-based model.

Graph classification. DepthGraphNet [26], an **Siamese GNN (SGNN)** is tailored to tackle the circuit graph isomorphism, specifically focusing on GLNs. By taking two circuit graphs as input, a GIN model extracts features from the graph through a sum aggregation. The extracted features by GIN are encoded using global sum pooling to produce a single encoding for each graph. The absolute difference of two graph encoding is calculated to capture the dissimilarity, passed through a fully connected layer to further predict if these two circuits are isomorphic. Compared to GNN-RE [13] using node-labeling techniques for subgraph detection, it involves two identical GNNs for feature extraction and comparison.

Representation learning. While the previously mentioned approaches rely solely on the structural properties of circuits for prediction, Wang *et al.* [195] introduce a framework based on CL that effectively captures logic functionality, proving both universal and transferable across various circuits. Moreover, the authors develop a functionality-centric GNN that works alongside this framework to perform classification tasks. This GNN extends the ABGNN [73] by utilizing independent aggregators for different types of nodes.

ConVERTS [52] proposes a self-supervised CL method to generalize netlist representation. It starts by converting RTL designs to AIGs and randomly synthesizes netlists using different recipes to augment the AIGs with equivalent functionality but varied structures. The augmented AIGs are encoded using its designed AIGNet, which aggregates feature information from neighboring nodes and repeats this process for multiple layers to capture information from a wider neighborhood. The method employs a CL scheme to make the cross-correlation matrix of graph embeddings close to the identity matrix, ensuring that each dimension of the embedding preserves information about netlist functionality but is invariant to structural changes. The effectiveness of ConVERTS is demonstrated in tasks like netlist classification and recovering the functionality of obfuscated designs, showing its ability to generalize well even for unseen designs.

3.3.4 Side-Channel Attacks.

Node classification. SCAR [174] is a pre-silicon framework to detect power side-channel vulnerabilities at RTL in cipher implementations. It converts RTL designs into CDFGs enriched with features such as connectivity, Hamming distance, and key dependency, then performs binary node classification to identify leaky modules. A GNN-Explainer maps these vulnerabilities to RTL lines, and a **large language model (LLM)** generates Boolean masking to remediate them.

Graph regression. **Physical design-aware power side-channel (PD-PSC)** leakage assessment [162] is a physical design-level framework that employs GCNs to predict dynamic power signatures from ASIC layouts. It extracts layout features from DEF, SPEF, and VCD files to construct a graph representation, and then uses a regression model to estimate power traces. These predictions are analyzed via Kullback–Leibler divergence to assess power side-channel vulnerability. The authors also provide an open-source dataset that comprises 100 distinct physical layout designs with 200,000 dynamic power traces (300 samples each) and detailed cell-level features.

3.3.5 Attacks on Design-for-Trust Methods. Numerous GNN-based techniques have achieved notable success in defeating logic locking and layout camouflaging. Its success either relies on the GNN’s abilities to learn IC netlist structures, although under the implementation of DfTr strategies, or consider the security measure architectures by directly learning, predicting, or bypassing themselves. [10, 14, 20, 22, 172]. These GNN-driven attacks are primarily executed in an oracle-less environment, which offers greater effectiveness compared to foundational attacks such as the Boolean **Satisfiability (SAT)**-based attack [176]. These state-of-the-art GNN methodologies have highlighted several vulnerabilities in the current DfTr mechanisms.

Node classification. GNNUnlock [10, 11], is a GNN model targeting **provably-secure logic locking (PSLL)**. Through the identified and isolated protection logic embedded within PSLL designs, GNNUnlock facilitates the restoration of the original design [116, 215].

Graph regression. The SAT-based attack, introduced by Subramanyan *et al.* [176], is a foundational attack on logic locking. However, its effectiveness is impacted by its dependable run-time based on the different circuit designs and its locking schemes, ranging from seconds to days. The ICNet platform [49] applies a GCN model to predict the run-time of the SAT-based attacks on specific circuits. It encodes each node/gate in the graph of locked netlist with a key-gate mask,

identifying whether it is a key-gate or a gate-type. Then, the trained models perform the regression task to predict the run-time.

Subgraph classification. OMLA [14], a GNN-based attack, targets X(N)OR locking by modeling the locked GLN as an undirected graph. It captures an h -hop enclosing subgraph around each key-gate, encompassing details about the key-gates and adjacent circuitry. By performing subgraph classification, OMLA is able to determine the values of the key bits.

MuxLink [22] and UNTANGLE [20] uncovered a novel vulnerability, link formation, that enables the attack of MUX-based logic locking [7, 88, 171], which was previously deemed resilient. The underlying rationale for MuxLink and UNTANGLE is the significant repetition and IP reuse inherent in modern ICs [161]. Consequently, both MuxLink and UNTANGLE utilize a GNN to learn the unlocked portions of a locked design and subsequently make predictions about the obfuscated interconnects. Specifically, these approaches transform the challenge of determining the inputs of a MUX key-gate into a link-prediction problem, which is then addressed using a GNN.

Titan [133], a GNN-based comprehensive attack platform for large-scale obfuscation, incorporates OMLA, MuxLink, and UNTANGLE methodologies to tackle gate and interconnect obfuscation. Titan enhances the GNN model to support additional classes and includes more target nodes to facilitate this comprehensive approach.

The effectiveness of GNN-based frameworks in detecting structural changes has been studied in depth. However, the capability of GNNs to determine graph isomorphism is upper bounded by the WL test [208]. The NetlistGNN [112] improves the ability of GNNs to reach the limit bounded by the heterogeneous WL test [208] through two variations: aggregating successor and predecessor nodes separately and stopping using attention for better generalization. By replacing the original GNN models in UNTANGLE [20] and MuxLink [22], the faulty key-bit values prediction gets reduced.

UN-SPLIT [18] attacks split manufacturing by modeling the missing BEOL interconnects for link prediction. The employed GNN considers the structure and composition of gates from the FEOL interconnects, along with physical design features, such as the placement and proximity of gates.

3.4 Discussion

3.4.1 Features Used. In the surveyed papers, diverse features are highlighted as critical for modeling circuits. Node features often include gate types, area, pin count, and power consumption to provide insights into the physical and electrical characteristics of circuit components. Further, state variables such as toggle rates and signal probabilities are utilized to capture dynamic behaviors and signal activity. Edge features are essential for detailing connections between nodes, with metrics like fan-in and fan-out, as well as timing data including delays, slews, and slack times, being commonly extracted. Geometrical features, such as distances to die boundaries and wire lengths, offer valuable information on the spatial layout of circuits. At the graph level, features such as hierarchical levels and clustering coefficients help represent the overall structure, while electrical parameters like capacitance and resistance provide detailed insights into the circuit's electrical properties.

3.4.2 Custom GNNs. Recently, a growing number of researchers have opted to introduce their custom GNN architecture. This change signals a positive shift in the understanding of GNNs and their potential to solve IC-related problems. For example, in [57], the custom GNN model, Circuit GNN, is able to learn on the custom graph representation introduced which embeds both topological and geometrical information. Previous studies (e.g., Net2 [207], CongestionNET [97]) that adopted popular GNNs such as GAT [186] failed to represent the interaction between cells and nets, which resulted in information loss and affected the performance of the downstream GNN. As for [232], the custom GNN overcomes the limitations of common GNNs by introducing a deeper model to

represent global graph information in the output nodes as compared to the previous models which have shallow receptive fields. This custom architecture solves the issue of decreasing regression performance by replacing supervised layers in the anterior layers of the model with MLPs to prevent over-smoothing. Moreover, the custom architecture restricts graph traversal order and message propagation scope at each iteration to reduce the computational overhead of GNN layers. Recall that other such custom architectures are identified in Tables 5, 6, and 7. We expect that, as the field develops, further custom and application-specific GNN architectures will be devised.

3.4.3 Hardware-Based Acceleration for GNNs. Hardware-based acceleration for GNNs addresses the computational complexity and memory demands of GNN algorithms [29, 45]. FPGA implementations can achieve substantial speedups compared to traditional CPU and GPU implementations, particularly for large-scale graph data, which is prevalent in IC design workflows. For instance, FPGA-based solutions have been developed to map graph processing tasks to hardware, leveraging parallelism and customizability to optimize execution time and energy consumption [45]. ASIC-based accelerators have shown improvements in throughput and power efficiency, making them suitable for data centers and edge devices with energy constraints. These accelerators handle the computational load of GNNs more effectively than general-purpose processors, achieving higher processing speeds and lower power consumption. Additionally, dedicated hardware units for GNN operations, such as specialized memory hierarchies and dataflow architectures, enhance the efficiency and scalability of these accelerators.

3.4.4 Selection of GNN Architectures. Across diverse tasks, different GNNs are chosen to match each problem's data structure and performance needs. GraphSAGE's [70] inductive, neighborhood-based aggregation excels on large or evolving netlists (e.g., congestion and slack prediction), while GCN's [96] normalized spectral formulation reliably captures global circuit properties like power and timing; GAT's [186] edge-wise attention highlights critical nets in routing, reliability, and trust analyses; MPNN's [66] rich edge attributes enable fine-grained delay and slew estimation; PNA's [54] multi-aggregator, degree-scaled design tailors to irregular timing-path aging predictions; GIN's [208] MLP-based aggregation distinguishes subtle subgraph structures for logic-block classification; and DGCNN's [225] provides canonical node ordering for link-prediction tasks in logic-locking attacks.

4 Future Directions

As we have demonstrated, GNNs have significant potential for IC design, reliability, and security. Several key challenges, however, remain. In this section, we outline key future directions for applying GNNs to IC design, specifically as they relate to scalability, data availability, explainability, domain integration, and hardware efficiency.

Scalability. Modern IC designs produce graphs with millions of nodes and edges, making both training and inference computationally intensive [3]. Future work should explore advanced graph sampling and partitioning techniques that allow training on smaller, more manageable subgraphs without losing critical information, as well as distributed and parallel processing approaches and algorithmic optimizations such as custom sparse-matrix operations and model pruning.

Data Availability. Another critical challenge is the limited availability of high-quality, labeled IC design data [118]. Proprietary restrictions and the labor-intensive nature of data labeling have resulted in a scarcity of comprehensive datasets, which hampers the development and generalization of robust GNN models [118]. To address this, future research must focus on generating realistic synthetic data and employing domain-specific data augmentation techniques. Establishing open benchmarks and utilizing transfer learning or self-supervised methods will also help improve model generalization across varied circuit designs.

Explainability. The explainability and trustworthiness of GNN models pose further challenges, especially in mission-critical applications where understanding model decisions is essential [218]. Integrating interpretable prediction techniques, such as attention mechanisms or saliency mapping, to highlight influential nodes and edges can enhance model transparency. Additionally, incorporating uncertainty estimation in model outputs can provide valuable insights into the confidence of predictions, thereby increasing reliability and fostering trust. Ensuring that model predictions conform to known physical and logical constraints is crucial for practical deployment.

Domain Integration. Integration of domain knowledge into GNN architectures and their seamless incorporation into traditional EDA toolflows is another important direction [195]. Hybrid models that combine data-driven learning with rule-based heuristics from established EDA tools can ensure that learned representations respect circuit design constraints. Developing standardized APIs and plug-in modules will facilitate the incorporation of GNN methods into existing design environments, and developing advanced heterogeneous GNN architectures tailored for diverse elements of analog and mixed-signal circuits is also crucial.

Hardware Efficiency. Efficient hardware implementation of GNN models is essential for real-time applications [45], [29]. As GNNs become integrated into live design and diagnostic systems, reducing latency and energy consumption through hardware acceleration will be critical. Exploiting FPGAs and ASICs to accelerate GNN computations can reduce latency and energy consumption for real-time applications. Techniques such as model compression, weight quantization, and incremental inference are also promising directions for making GNN computations more hardware-friendly. Future research should focus on co-designing algorithms and hardware to meet the stringent requirements of real-world applications.

Noise Sensitivity. While our survey highlights process variation and aging as reliability concerns, many GNN-EDA models are trained and evaluated solely on clean, corner-case data, without any assessment of noise, be it temperature fluctuations, power-supply jitter, measurement errors, and so on. Consequently, these models may fall short when deployed under real-world settings [85]. Bridging this gap will require (i) Monte-Carlo or adversarial testing suites that stress-test GNN predictions under calibrated noise [177], (ii) incorporation of Bayesian or uncertainty-aware GNN layers to quantify predictive confidence [188], [44], and (iii) noise-augmented training protocols, e.g., data perturbation or adversarial augmentation, all to harden models accordingly.

Layout-Dependent Effects. Many GNN frameworks operate on abstract netlists or coarse placement statistics (e.g., cell coordinates, metal-layer counts, via counts, simple wire resistance) and omit the post-layout parasitic extraction data (e.g., capacitance C , resistance R , inductance L) that commercial sign-off tools compute after routing. This omission limits the path-delay prediction accuracy, and prevents the models from flagging power-integrity issues (e.g., IR-drop, switching noise) or signal-integrity hazards such as coupling-induced glitches and reflections. To overcome these limitations, future work may inject extracted $R/C/L$ values as node and edge features and benchmark on public datasets pairing routed layouts with complete SPEF/SDF decks [1, 2].

5 GNN4CIRCUITS

5.1 Overview and Motivation

GNN4CIRCUITS is an end-to-end Python-based tool that transforms hardware circuit designs described in Verilog or Bench format into graph representations and supports various GNN-based learning tasks that aim to effectively classify and predict circuit characteristics. It is freely available at [<https://github.com/DfX-NYUAD/GNN4CIRCUITS>].

The creation of the *GNN4CIRCUITS* tool came from a need to streamline the process of transforming hardware circuit designs into graph representations for downstream GNN-based learning tasks. Many research projects implement their own parser and GNN architecture, which can be

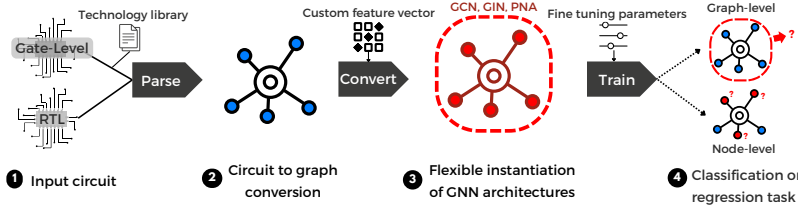


Fig. 7. GNN4CIRCUITS pipeline visualization.

inefficient when created separately for each dataset and task at hand. This tool aims to eliminate the need for a new hard-coded implementation of a parser-converter that utilizes GNNs with each research project. It also allows researchers to perform most of the experiments presented in the survey section without requiring an extensive background in coding ML and GNN algorithms. The tool supports different types of circuit representations, and unfixed hyperparameters and GNN architectures. Rather, they can be determined by the user, thus allowing customization and application to a wider set of tasks. By setting parameters, users can utilize this tool to classify and predict circuit characteristics, enabling researchers with different backgrounds to engage with advanced ML techniques in hardware design.

The general pipeline can be broken down into (1) parsing, (2) converting, and (3) training, visualized in Figure 7. (1) parsing: *GNN4CIRCUITS* supports the transformation of textual hardware circuit designs in the GLN, RTL and bench format. *GNN4CIRCUITS* traverses any given work directory, parses all files, and keeps an array of related graphs. Depending on the type of hardware design, it follows the respective parsing steps. (2) Converting: then, the NetworkX Python package [69] is utilized to format the graphs. It provides rich graph data structures, comprehensive graph algorithms and easy conversion to **Deep Graph Library (DGL)** [194], a Python package built for DL on graphs, used for the ML portion of the tool. *GNN4CIRCUITS* can be used for hardware to graph conversion, as well as for graph and node classification and regression tasks.

GNN4CIRCUITS contributes to the field of EDA by leveraging graph representations and ML models to analyze and predict properties of ICs. (3) Training: this tool provides a customizable and automated experience for researchers without the necessary background in ML, and eliminates the need for new hard-coded implementations with each research project. *GNN4CIRCUITS* can be used to reproduce the results from the articles presented in this survey. To demonstrate its versatility, we replicated experiments from three representative works: Wu et al. [204] for EDA tasks, GNN4REL [16] for hardware reliability, and GNN-RE [13] for hardware security. Note that our primary goal is to provide a unified tool for applying GNN models to diverse tasks in IC design, including EDA, reliability, and security analyses. The tool enables graph extraction from circuit designs and supports a variety of GNN-based downstream tasks, demonstrating versatility and adaptability for integration into IC design flows. Regarding hyperparameters and model configurations, our focus is on aligning with commonly used practices in prior works rather than proposing new architectures or fine-tuning performance. We therefore retain the original model structures and settings reported in these studies to ensure consistency. Additionally, we implemented and evaluated widely used GNN models (GIN, GCN, and PNA), further showcasing the tool's ability to support multiple architectures and hardware analysis scenarios.

5.2 Case Studies

5.2.1 Electronic Design Automation. Using *GNN4CIRCUITS*, we reproduced the node-level resource classification by Wu et al. [204], which is an EDA-focused GNN problem. By adopting the original experimental setup and hyperparameters (i.e., five GNN layers with 300-dimensional

Table 2. GNN4CIRCUITS Applies to EDA: Node Classification Accuracy on DFGs, CDFGs, and Real Circuits

	DFG			CDFG			Real Circuits		
	DSP (%)	LUT (%)	FF (%)	DSP (%)	LUT (%)	FF (%)	DSP (%)	LUT (%)	FF (%)
GCN	93.79	84.84	88.66	83.00	77.01	64.74	79.70	81.83	86.82
SAGE	93.06	87.32	92.09	86.55	78.41	67.40	79.39	86.44	55.88
GIN	93.80	84.93	91.57	79.24	73.25	65.78	74.70	75.53	72.24
RGCN	93.91	87.13	91.52	85.80	78.46	68.27	90.82	88.83	91.55

(a) Results from four different GNN models by Wu *et al.* [204]

	DFG			CDFG			Real Circuits		
	DSP (%)	LUT (%)	FF (%)	DSP (%)	LUT (%)	FF (%)	DSP (%)	LUT (%)	FF (%)
GCN	96.14	93.53	90.03	95.52	90.56	87.98	98.15	96.87	95.75
GIN	98.25	98.46	95.43	98.39	98.23	96.43	92.56	92.53	91.21
PNA	93.58	92.41	89.06	96.23	95.75	92.40	93.61	93.52	92.00

(b) Results from three different GNN models by GNN4CIRCUITS.

hidden embeddings, sum/mean pooling, the Adam optimizer, and 300 training epochs), we obtained classification accuracies on synthetic DFGs, CDFGs generated by ldrngen [37], and real-world HLS benchmarks from MachSuite [155], CHStone [71], and PolyBench/C [146]. We implemented the three available GNN architectures (GCN [96], GIN [96], and PNA [54]) within GNN4CIRCUITS.

Table 2(b) presents the GNN4CIRCUITS classification accuracies on resources types (DSP, LUT, and FF). The tool parses and extracts graph representations from DFGs, CDFGs, and real-case circuit designs and train models with three GNN variants for node classification on different resource types. As shown in Table 2(b), both GIN and GCN perform well on the current case studies, and for reference, we also include the original results in Table 2(a). The accuracies are high and closely match those reported in the original study [204]. This validates GNN4CIRCUITS' ability to emulate established GNN-based EDA workflows such as those presented in Section 3.1.

5.2.2 Hardware Reliability. We reproduced the path-level regression for process-variation degradation as presented in GNN4REL [16], extracting one-hop subgraphs around each timing path in select EPFL benchmarks [25], for 500 epochs using Adam optimizer with a learning rate of 0.001, hidden dimension of 64, and a batch size of 32. Where GNN4REL [16] originally employs a PNA-based regressor [54], we implemented three GNN variants - GCN [96], GIN [96], and PNA [54]. Table 3 presents the edge-level regression results using the MAE metric. As GNN4REL reports its regression performance using MAE and MAPE, we adopt the MAE metric here to evaluate the regression performance of GNN4CIRCUITS in a consistent manner. PNA achieves the best performance on the adder and multiplier benchmarks, while GIN performs best on the max benchmark. These results highlight the ability of GNN4CIRCUITS to parse subgraphs, train GNNs, and deliver accurate predictions for hardware reliability tasks such as those discussed in Section 3.2.

5.2.3 Hardware Security. We reproduced the node-classification experiments from GNN-RE [13] using the interconnected-modules dataset within the GNN4CIRCUITS framework. Following the original experimental configuration, we employed a five-layer GNN architecture and 256 hidden dimensions. Training followed the original setup, optimized with Adam for up to 2,000 epochs. We evaluated three GNN models: GCN [96], GIN [96], and PNA [54]. Table 4 summarizes the node classification results on RE. It extracts circuit graphs and offers three variant GNN models, where GIN [96] achieves the highest accuracy to predict the circuit functionality. Our results further

Table 3. GNN4CIRCUITS Applies to Hardware Reliability: MAE of Edge Regression for Average Process Variation Degradation on Selected EPFL Benchmarks under the Self-Referencing Scenario

	adder	multiplier	max
GNN4REL: PNA	0.22	0.02	0.06
GNN4CIRCUITS: GCN	0.43	0.03	0.11
GNN4CIRCUITS: GIN	0.30	0.03	0.07
GNN4CIRCUITS: PNA	0.25	0.02	0.08

Table 4. GNN4CIRCUITS Applies to Hardware Security: Node Classification Performance for GLN RE

	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
GNN-RE: SAGE	98.87	94.74–100	94.88–99.61	96.1–99.38
GNN4CIRCUITS: GCN	87.94	68.51	87.43	73.08
GNN4CIRCUITS: GIN	91.15	74.59	78.38	73.46
GNN4CIRCUITS: PNA	76.97	37.48	41.05	34.21

validate the effectiveness and adaptability of GNN4CIRCUITS in addressing circuit-level security tasks, in line with other tasks evaluated in our broader survey.

5.2.4 Summary. These successful experiments confirm that GNN4CIRCUITS can serve as a plug-and-play framework for a variety of tasks, including EDA, reliability, and security tasks, while highlighting its efficacy and practical relevance with the rest of the survey. Its contributions and innovation lie in providing a generalized and highly customizable tool that would allow researchers of different backgrounds to apply ML to various IC-based tasks.

5.3 Limitations

While GNN4CIRCUITS offers a flexible and extensible framework for applying GNNs to circuit analysis tasks, several limitations remain:

5.3.1 Node-Level Labeling Challenges. A primary limitation lies in the support for node-level learning tasks. For graph-level classification, the platform allows users to organize Verilog files into label-based directories, enabling automatic graph labeling. However, this mechanism does not generalize to node-level problems, as Verilog files lack a consistent and generalizable way to assign labels to individual nodes. Consequently, node-level tasks require a separate data pipeline. GNN4CIRCUITS addresses this by providing a `parse_txt()` function that accepts structured text inputs (`cell.txt`, `feat.txt`, `link.txt`, `paths.txt`, and `label.txt`) for defining node features and labels. While effective, this approach relies on users to preprocess and format their data manually, which may hinder adoption by non-experts.

5.3.2 Lack of a Graphical User Interface (GUI). GNN4CIRCUITS currently operates as a script-based tool with no graphical interface, which can limit usability for those without coding expertise. A GUI could broaden accessibility and streamline tasks such as data input, model configuration, and result visualization.

6 Conclusions

We have conducted an in-depth survey of GNN applications across multiple IC design domains, classifying state-of-the-art works into three key categories: GNNs for EDA, GNNs for reliable ICs, and GNNs for hardware security. The related references, datasets, codes, and resources are

summarized in our GNN4IC hub at [<https://github.com/DfX-NYUAD/GNN4IC>]. We have also conducted exemplary case studies for these domains using our proposed GNN4CIRCUITS tool, available at [<https://github.com/DfX-NYUAD/GNN4CIRCUITS>].

Although GNNs are well-suited for learning on circuits and show significant potential, their application brings specific challenges. For example, several works have required custom GNN architectures and employed edge/node features or weights to ensure model expressiveness while preserving the inherent directionality of circuits [13, 132, 184]. Other studies have used heterogeneous GNN architectures to handle the diverse node types present in analog circuits.

Previously, much of the research has concentrated on structural aspects of circuits, often overlooking functional properties. However, recent studies demonstrate that incorporating functional features can significantly improve GNN performance [195, 230]. Scalability has also been a critical focus, with many prior works utilizing graph sampling or clustering to accelerate training [10, 13]. Additionally, more specific solutions have been developed, such as custom matrix multiplication to avoid repetitive computations [132] and embedding design rules directly into the model [223].

Among the surveyed domains, GNNs for reliability represent a relatively new area, which we expect to grow in the near future. GNNs for EDA, in contrast, are more mature and have investigated various graph types, architectures, and tasks. For hardware security, GNNs have primarily been used to evaluate the robustness of DfTr methods, though their potential to enhance IC design security remains largely untapped. Going forward, we expect the integration of trustworthy ML and explainable AI concepts to have a positive impact on these applications.

References

- [1] 2000. IEEE standard for integrated circuit (IC) delay and power calculation system. *IEEE Std 1481-1999* (2000), 1–400. DOI: <https://doi.org/10.1109/IEEESTD.2000.7395044>
- [2] 2001. IEEE standard for standard delay format (SDF) for the electronic design process. *IEEE Std 1497-2001* (2001), 1–80. DOI: <https://doi.org/10.1109/IEEESTD.2001.93359>
- [3] S. Abadal, A. Jain, R. Guirado, J. López-Alonso, and E. Alarcón. 2021. Computing graph neural networks: A survey from algorithms to accelerators. *ACM Computing Surveys* 54, 9 (2021), 1–38.
- [4] M. Agarwal, B. C. Paul, M. Zhang, and S. Mitra. 2007. Circuit failure prediction and its application to transistor aging. In *Proceedings of the VTS*.
- [5] A. Agnesina, K. Chang, and S. K. Lim. 2020. A general framework for VLSI tool parameter optimization with deep reinforcement learning. In *Proceedings of the NeurIPS*.
- [6] A. Agnesina, K. Chang, and S. K. Lim. 2020. VLSI placement parameter optimization using deep reinforcement learning. In *Proceedings of the ICCAD*.
- [7] A. Alaql, M. M. Rahman, and S. Bhunia. 2021. SCOPE: Synthesis-based constant propagation attack on logic locking. *TVLSI* 29, 8 (2021), 1529–1542.
- [8] Y. Alkabani, F. Koushanfar, and M. Potkonjak. 2007. Remote activation of ICs for piracy prevention and digital right management. In *Proceedings of the ICCAD*.
- [9] L. Alrahis, M. Yasin, H. Saleh, B. Mohammad, and M. A. Qutayri. 2019. Functional reverse engineering on SAT-attack resilient logic locking. In *Proceedings of the ISCAS*.
- [10] L. Alrahis, S. Patnaik, M. A. Hanif, H. Saleh, M. Shafique, and O. Sinanoglu. 2021. GNNUnlock+: A systematic methodology for designing graph neural networks-based oracle-less unlocking schemes for provably secure logic locking. *TETC* 10, 3 (2021), 1575–1592.
- [11] L. Alrahis, S. Patnaik, F. Khalid, M. A. Hanif, H. Saleh, M. Shafique, and O. Sinanoglu. 2021. GNNUnlock: Graph neural networks-based oracle-less unlocking scheme for provably secure logic locking. In *Proceedings of the DATE*.
- [12] L. Alrahis, S. Patnaik, J. Knechtel, H. Saleh, B. Mohammad, M. A. Qutayri, and O. Sinanoglu. 2021. UNSAIL: Thwarting oracle-less machine learning attacks on logic locking. *TIFS* 16 (2021), 2508–2523.
- [13] L. Alrahis, A. Sengupta, J. Knechtel, S. Patnaik, H. Saleh, B. Mohammad, M. A. Qutayri, and O. Sinanoglu. 2022. GNN-RE: Graph neural networks for reverse engineering of gate-level netlists. *TCAD* 41, 8 (2022), 2435–2448.
- [14] L. Alrahis, S. Patnaik, M. Shafique, and O. Sinanoglu. 2022. OMLA: An oracle-less machine learning-based attack on logic locking. *TCAS-II* 69, 3 (2022), 1602–1606.
- [15] L. Alrahis, S. Patnaik, M. A. Hanif, M. Shafique, and O. Sinanoglu. 2023. PoisonedGNN: Backdoor attack on graph neural networks-based hardware security systems. *TC* 72, 10 (2023), 2822–2834.

- [16] L. Alrahis, J. Knechtel, F. Klemme, H. Amrouch, and O. Sinanoglu. 2022. GNN4REL: Graph neural networks for predicting circuit reliability degradation. *TCAD* 41, 11 (2022), 3826–3837.
- [17] L. Alrahis, J. Knechtel, and O. Sinanoglu. 2023. Graph neural networks: A powerful and versatile tool for advancing design, reliability, and security of ICs. In *Proceedings of the ASP-DAC*.
- [18] L. Alrahis, L. Mankali, S. Patnaik, A. Sengupta, J. Knechtel, and O. Sinanoglu. 2023. UN-SPLIT: Attacking split manufacturing using link prediction in graph neural networks. In *Proceedings of the SPACE*.
- [19] L. Alrahis, H. Nassar, J. Krautter, D. Gnad, L. Bauer, J. Henkel, and M. Tahoori. 2024. MaliGNNoma: GNN-based malicious circuit classifier for secure cloud FPGAs. In *Proceedings of the HOST*.
- [20] L. Alrahis, S. Patnaik, M. A. Hanif, M. Shafique, and O. Sinanoglu. 2021. UNTANGLE: Unlocking routing and logic obfuscation using graph neural networks-based link prediction. In *Proceedings of the ICCAD*.
- [21] L. Alrahis, S. Patnaik, M. Shafique, and O. Sinanoglu. 2022. Embracing graph neural networks for hardware security. In *Proceedings of the ICCAD*.
- [22] L. Alrahis, S. Patnaik, M. Shafique, and O. Sinanoglu. 2022. MuxLink: Circumventing learning-resilient MUX-locking using graph neural network-based link prediction. In *Proceedings of the DATE*.
- [23] L. Alrahis, M. Yasin, N. Limaye, H. Saleh, B. Mohammad, M. Alqutayri, and O. Sinanoglu. 2019. ScanSAT: Unlocking static and dynamic scan obfuscation. *TETC* 9, 4 (2019), 1867–1882.
- [24] L. Alrahis, M. Yasin, H. Saleh, B. Mohammad, M. Al-Qutayri, and O. Sinanoglu. 2019. ScanSAT: Unlocking obfuscated scan chains. In *Proceedings of the ASP-DAC*.
- [25] L. Amaru, P. Gaillardon, and G. De Micheli. 2015. The EPFL combinational benchmark suite. In *Proceedings of the IWLS*.
- [26] F. Amin, S. Chatterjee, and P. D. Franzon. 2023. DepthGraphNet: Circuit graph isomorphism detection via siamese-graph neural networks. In *Proceedings of the MLCAD*.
- [27] S. Amizadeh, S. Matuskevych, and M. Weimer. 2018. Learning to solve circuit-sat: An unsupervised differentiable approach. In *Proceedings of the ICLR*.
- [28] H. Amrouch, B. Khaleghi, A. Gerstlauer, and J. Henkel. 2016. Reliability-aware design to suppress aging. In *Proceedings of the DAC*.
- [29] A. Auten, M. Tomei, and R. Kumar. 2020. Hardware acceleration of graph neural networks. In *Proceedings of the DAC*.
- [30] K. Z. Azar, M. M. Hossain, F. Rahman, F. Farahmandi, and M. M. Tehranipoor. 2022. Fuzz, Penetration, and AI Testing for SoC Security Verification: Challenges and Solutions. *Cryptol. ePrint*. (2022).
- [31] K. Baek, H. Park, S. Kim, K. Choi, and T. Kim. 2022. Pin accessibility and routing congestion aware DRC hotspot prediction using graph neural network and U-Net. In *Proceedings of the ICCAD*.
- [32] X. Bai, C. Visweswariah, and P. N. Strenski. 2002. Uncertainty-aware circuit optimization. In *Proceedings of the DAC*.
- [33] A. Balakrishnan, T. Lange, M. Glorieux, D. Alexandrescu, and M. Jenihhin. 2019. Modeling gate-level abstraction hierarchy using graph convolutional neural networks to predict functional de-rating factors. In *Proceedings of the AHS*.
- [34] A. Balakrishnan, T. Lange, M. Glorieux, D. Alexandrescu, and M. Jenihhin. 2020. Composing graph theory and deep neural networks to evaluate SEU type soft error effects. In *Proceedings of the MECCO*.
- [35] N. Banerjee, G. Karakonstantis, and K. Roy. 2007. Process variation tolerant low power DCT architecture. In *Proceedings of the DATE*. 49–57.
- [36] C. Bao, D. Forte, and A. Srivastava. 2015. On reverse engineering-based hardware Trojan detection. *TCAD* 35, 1 (2015).
- [37] G. Barany. 2017. Liveness-driven random program generation. In *Proceedings of the LOPSTR*.
- [38] S. Bhunia, M. S. Hsiao, M. Banga, and S. Narasimhan. 2014. Hardware trojan attacks: Threat analysis and countermeasures. *Proceedings of the IEEE* 102, 8 (2014), 1229–1247.
- [39] T. Bian, X. Xiao, T. Xu, P. Zhao, W. Huang, Y. Rong, and J. Huang. 2020. Rumor detection on social media with bi-directional graph convolutional networks. In *Proceedings of the AAAI*, Vol. 34.
- [40] B. E. Boser, I. M. Guyon, and V. N. Vapnik. 1992. A training algorithm for optimal margin classifiers. In *Proceedings of the COLT*.
- [41] T. Bucher, L. Alrahis, G. Paim, S. Bampi, O. Sinanoglu, and H. Amrouch. 2022. AppGNN: Approximation-aware functional reverse engineering using graph neural networks. *ICCAD* (2022), 1–9.
- [42] H. Chen, K. C. Hsu, W. J. Turner, P. H. Wei, K. Zhu, D. Z. Pan, and H. Ren. 2023. Reinforcement learning guided detailed routing for custom circuits. In *Proceedings of the ISPD*.
- [43] H. Chen, K. Zhu, M. Liu, X. Tang, N. Sun, and D. Z. Pan. 2021. Universal symmetry constraint extraction for analog and mixed-signal circuits with graph neural networks. In *Proceedings of the DAC*.
- [44] Qingfeng Chen, Shiyuan Li, Yixin Liu, Shirui Pan, Geoffrey I. Webb, and Shichao Zhang. 2025. Uncertainty-Aware Graph Neural Networks: A Multi-Hop Evidence Fusion Approach. *IEEE Transactions on Neural Networks and Learning Systems*. DOI: [10.1109/TNNLS.2025.3581083](https://doi.org/10.1109/TNNLS.2025.3581083)
- [45] S. Chen, J. Liu, and L. Shen. 2024. A survey on graph neural network acceleration: A hardware perspective. *Chinese Journal of Electronics* 33, 3 (2024), 601–622.

- [46] T. Chen, H. Geng, Q. Sun, S. Wan, Y. Sun, H. Yu, and B. Yu. 2021. Analog IC aging-induced degradation estimation via heterogeneous graph convolutional networks. In *Proceedings of the ASP-DAC*.
- [47] T. Chen, Q. Sun, C. Zhan, C. Liu, H. Yu, and B. Yu. 2022. Deep H-GCN: Fast analog IC aging-induced degradation estimation. *TCAD* 41, 7 (2022), 1990–2003.
- [48] X. Chen, Z. Di, W. Wu, Q. Wu, J. Shi, and Q. Feng. 2022. Detailed routing short violation prediction using graph-based deep learning model. *TCAS-II* 69, 2 (2022), 564–568.
- [49] Z. Chen, G. Kolhe, S. Rafatirad, Sai Manoj P. D., H. Homayoun, L. Zhao, and C. T. Lu. 2020. Estimating the circuit de-obfuscation runtime based on graph deep learning. In *Proceedings of the DATE*. 101822.
- [50] D. Cheng, C. Dong, W. He, Z. Chen, X. Liu, and H. Zhang. 2023. A fine-grained detection method for gate-level hardware Trojan base on bidirectional graph neural networks. *Journal of King Saud University - Computer and Information Sciences* 35, 10 (2023), 16508–16519.
- [51] R. Cheng and J. Yan. 2021. On Joint Learning for Solving Placement and Routing in Chip Design. *NeurIPS* 34 (2021).
- [52] A. B. Chowdhury, J. Bhandari, L. Collini, R. Karri, B. Tan, and S. Garg. 2023. ConVERTS: Contrastively learning structurally invariant netlist representations. In *Proceedings of the MLCAD*.
- [53] S. D. Chowdhury, K. Yang, and P. Nuzzo. 2021. ReIGNN: State register identification using graph neural networks for circuit reverse engineering. In *Proceedings of the ICCAD*.
- [54] G. Corso, L. Cavalleri, D. Beaini, P. Liò, and P. Velickovic. 2020. Principal neighbourhood aggregation for graph nets. *NeurIPS* 33 (2020), 13260–13271.
- [55] A. Cui, G. Qu, and Y. Zhang. 2015. Ultra-low overhead dynamic watermarking on scan design for hard IP protection. *TIFS* 10, 11 (2015), 2298–2313.
- [56] S. De, M. Shafique, and H. Corporaal. 2023. Delay prediction for ASIC HLS: Comparing graph-based and nongraph-based learning models. *TCAD* 42, 4 (2023), 1133–1146.
- [57] Z. Dong, W. Cao, M. Zhang, D. Tao, Y. Chen, and X. Zhang. 2023. CktGNN: Circuit graph neural network for electronic design automation. In *Proceedings of the ICLR*.
- [58] EBSCO. 2023. Retrieved from <https://www.ebsco.com/products/ebsco-discovery-service>.
- [59] R. Elnaggar and K. Chakrabarty. 2018. Machine learning for hardware security: Opportunities and risks. *JETTA* (2018).
- [60] M. Ferianc, H. Fan, R. S. W. Chu, J. Stano, and W. Luk. 2020. Improving performance estimation for FPGA-based accelerators for convolutional neural networks. In *Proceedings of the ARC*.
- [61] L. Ferretti, A. Cini, G. Zacharopoulos, C. Alippi, and L. Pozzi. 2022. Graph neural networks for high-level synthesis design space exploration. *TODAES* 28, 2 (2022), 1–20.
- [62] E. Fix. 1985. *Discriminatory Analysis: Nonparametric Discrimination, Consistency Properties*. Vol. 1.
- [63] G. Flach, T. Reimann, G. Posser, M. Johann, and R. Reis. 2014. Effective method for simultaneous gate sizing and vth assignment using lagrangian relaxation. *TCAD* 33, 4 (2014), 546–557.
- [64] M. Fojtik, D. Fick, Y. Kim, N. Pinckney, D. Harris, D. Blaauw, and D. Sylvester. 2012. Bubble razor: An architecture-independent approach to timing-error detection and correction. In *Proceedings of the ISSCC*.
- [65] A. Ghose, V. Zhang, Y. Zhang, D. Li, W. Liu, and M. Coates. 2021. Generalizable cross-graph embedding for GNN-based congestion prediction. In *Proceedings of the ICCAD*.
- [66] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. 2017. Neural message passing for quantum chemistry. In *Proceedings of the ICML*.
- [67] A. Grover and J. Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the KDD*.
- [68] Z. Guo, M. Liu, J. Gu, S. Zhang, D. Z. Pan, and Y. Lin. 2022. A timing engine inspired graph neural network model for pre-routing slack prediction. In *Proceedings of the DAC*.
- [69] A. A. Hagberg, D. A. Schult, and P. J. Swart. 2008. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the SciPy*.
- [70] W. Hamilton, Z. Ying, and J. Leskovec. 2017. Inductive representation learning on large graphs. In *Proceedings of the NIPS*. 242–254.
- [71] Y. Hara, H. Tomiyama, S. Honda, and H. Takada. 2009. Proposal and quantitative analysis of the CHStone benchmark program suite for practical C-Based high-level synthesis. *Journal of Information Processing* 17 (2009).
- [72] R. Hassan, X. Meng, K. Basu, and S. M. P. Dinakar Rao. 2023. Circuit topology-aware vaccination-based hardware trojan detection. *TCAD* 42, 9 (2023), 2852–2862.
- [73] Z. He, Z. Wang, C. Bail, H. Yang, and B. Yu. 2021. Graph learning-based arithmetic block identification. In *Proceedings of the ICCAD*. 37–45.
- [74] S. Hochreiter and J. Schmidhuber. 1997. Long short-term memory. *Neural Comput.* 9, 8 (1997).
- [75] X. Hong, Y. Y. Tee, T. Lin, Y. Shi, D. Cheng, E. Huang, and B. H. Gwee. 2023. GoCLIP: Graph one-class classification for intellectual property circuit identification. In *Proceedings of the IPFA*.
- [76] X. Hong, T. Lin, Y. Shi, and B. H. Gwee. 2023. GraphClusNet: A hierarchical graph neural network for recovered circuit netlist partitioning. *TAI* 4, 5 (2023), 1199–1213.

- [77] X. Hong, T. Lin, Y. Shi, and B. H. Gwee. 2021. ASIC circuit netlist recognition using graph neural network. In *Proceedings of the IPFA*. 21415–21428.
- [78] Y. Hu, J. Li, F. Klemme, G. J. Nam, T. Ma, H. Amrouch, and J. Xiong. 2024. SyncTREE: Fast timing analysis for integrated circuit design through a physics-informed tree-based graph neural network. *NeurIPS* 36 (2024).
- [79] G. Huang, J. Hu, Y. He, J. Liu, M. Ma, Z. Shen, J. Wu, Y. Xu, H. Zhang, K. Zhong, X. Ning, Y. Ma, H. Yang, B. Yu, H. Yang, and Y. Wang. 2021. Machine learning for electronic design automation: A survey. *TODAES* 26, 5 (2021), 1–46.
- [80] G. Huang, J. Hu, Y. He, J. Liu, M. Ma, Z. Shen, J. Wu, Y. Xu, H. Zhang, K. Zhong, X. Ning, Y. Ma, H. Yang, B. Yu, H. Yang, and Y. Wang. 2021. Machine learning for electronic design automation: A survey. *TODAES* 26, 5 (2021), 1–46.
- [81] Z. Huang, Q. Wang, Y. Chen, and X. Jiang. 2020. A survey on machine learning against hardware trojan attacks: Recent advances and challenges. *IEEE Access* 8 (2020), 10796–10826.
- [82] S. Hung, S. Banerjee, A. Chaudhuri, J. Kim, S. K. Lim, and K. Chakrabarty. 2023. Transferable graph neural network-based delay-fault localization for monolithic 3D ICs. *TCAD* (2023), 448–453.
- [83] A. Hussein, M. M. Gaber, E. Elyan, and C. Jayne. 2017. Imitation learning: A survey of learning methods. 50, 2, Article 21 (2017), 1–35.
- [84] W. Jiang and J. Luo. 2022. Graph neural network for traffic forecasting: A survey. *Expert Systems with Applications* 207 (2022), 117921.
- [85] Wei Ju, Siyu Yi, Yifan Wang, Zhiping Xiao, Zhengyang Mao, Hourun Li, Yiyang Gu, Yifang Qin, Nan Yin, Senzhang Wang, X. Liu, X. Luo, P. S. Yu, and M. Zhang. 2024. A Survey of Graph Neural Networks in Real world: Imbalance, Noise, Privacy and OOD Challenges. (2024). arXiv:2403.04468. Retrieved from <https://arxiv.org/abs/2403.04468>
- [86] L. P. Kaelbling, M. L. Littman, and A. W. Moore. 1996. Reinforcement learning: A survey. *JAIR* 4 (1996), 237–285.
- [87] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu. 2011. *VLSI physical design: From graph partitioning to timing closure*. Vol. 312.
- [88] H. M. Kamali, K. Z. Azar, H. Homayoun, and A. Sasan. 2020. InterLock: An intercorrelated logic and routing locking. In *Proceedings of the ICCAD*.
- [89] H. M. Kamali, K. Z. Azar, F. Farahmandi, and M. Tehranipoor. 2022. Advances in Logic Locking: Past, Present, and Prospects. *Cryptol. ePrint*. (2022).
- [90] R. Karri, J. Rajendran, K. Rosenfeld, and M. Tehranipoor. 2010. Trustworthy hardware: Identifying and classifying hardware trojans. *Computer* 43, 10 (2010), 39–46.
- [91] B. Khailany, H. Ren, S. Dai, S. Godil, B. Keller, R. Kirby, A. Klinefelter, R. Venkatesan, Y. Zhang, B. Catanzaro, and W. J. Dally. 2020. Accelerating chip design with machine learning. *Micro* 40, 6 (2020), 33–33.
- [92] K. Khalil, O. Eldash, A. Kumar, and M. Bayoumi. 2020. Machine learning-based approach for hardware faults prediction. *TCAS-I* 67, 11 (2020), 3880–3892.
- [93] S. Khan, Z. Shi, M. Li, and Q. Xu. 2024. DeepSeq: Deep sequential circuit learning. In *Proceedings of the DATE*.
- [94] J. Kim, H. Kim, H. Amrouch, J. Henkel, A. Gerstlauer, and K. Choi. 2019. Aging gracefully with approximation. In *Proceedings of the ISCAS*.
- [95] K. K. Kim, W. Wang, and K. Choi. 2010. On-chip aging sensor circuits for reliable nanometer MOSFET digital circuits. *TCAS-II* 57, 10 (2010), 798–802.
- [96] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the ICLR*.
- [97] R. Kirby, S. Godil, R. Roy, and Bryan Catanzaro. 2019. CongestionNet: Routing congestion prediction using deep graph neural networks. In *Proceedings of the VLSI-Soc*.
- [98] F. Klemme and H. Amrouch. 2021. Machine learning for circuit aging estimation under workload dependency. In *Proceedings of the ITC*.
- [99] J. Knechtel. 2020. Hardware security for and beyond CMOS technology: An overview on fundamentals, applications, and challenges. In *Proceedings of the ISPD*.
- [100] J. Knechtel, E. B. Kavun, F. Regazzoni, A. Heuser, A. Chattopadhyay, D. Mukhopadhyay, S. Dey, Y. Fei, Y. Belenky, I. Levi, T. Güneş, P. Schaumont, and I. Polian. 2020. Towards secure composition of integrated circuits and electronic systems: On the role of EDA. In *Proceedings of the DATE*.
- [101] J. Knechtel, S. Patnaik, and O. Sinanoglu. 2019. Protect your chip design intellectual property: An overview. In *Proceedings of the IOLTS*.
- [102] P. Kocher, J. Jaffe, and B. Jun. 1999. Differential power analysis. In *Proceedings of the CRYPTO*.
- [103] A. Koufopoulou, A. Papadimitriou, A. Pikrakis, M. Psarakis, and D. Hely. 2023. On the prediction of hardware security properties of HLS designs using graph neural networks. In *Proceedings of the DFT*.
- [104] R. Kumar and V. Kursun. 2006. Reversed temperature-dependent propagation delay characteristics in nanometer CMOS circuits. *TCAS-II* 53, 10 (2006), 1078–1082.
- [105] K. Kunal, T. Dhar, M. Madhusudan, J. Poojary, A. Sharma, W. Xu, S. M. Burns, J. Hu, R. Harjani, and S. S. Sapatnekar. 2020. GANA: Graph convolutional network based automated netlist annotation for analog circuits. In *Proceedings of the DATE*.

- [106] K. Kunal, T. Dhar, M. Madhusudan, J. Poojary, A. K. Sharma, W. Xu, S. M. Burns, J. Hu, R. Harjani, and S. S. Sapatnekar. 2023. GNN-Based hierarchical annotation for analog circuits. *TCAD* 42, 9 (2023), 2801–2814.
- [107] Masanori Kurimoto, Hiroaki Suzuki, Rei Akiyama, Tadao Yamanaka, Haruyuki Ohkuma, Hidehiro Takata, and Hirofumi Shinohara. 2010. Phase-adjustable error detection flip-flops with 2-stage hold-driven optimization, slack-based grouping scheme and slack distribution control for dynamic voltage scaling. *TODAES* 15, 2 (2010), 1–17.
- [108] Y. LeCun, Y. Bengio, and G. Hinton. 2015. Deep learning. *Nature* 521, 7553 (2015), 436–444.
- [109] J. W. Lee, T. Dhar, M. Madhusudan, J. Poojary, A. K. Sharma, W. Xu, S. M. Burns, J. Hu, R. Harjani, and S. S. Sapatnekar. 2022. A learning-based algorithm for early floorplan with flexible blocks. In *Proceedings of the A-SSCC*.
- [110] L. Li and A. Orailoglu. 2019. Piercing logic locking keys through redundancy identification. In *Proceedings of the DATE*.
- [111] M. Li, S. Khan, Z. Shi, N. Wang, H. Yu, and Q. Xu. 2022. Deepgate: Learning neural representations of logic gates. In *Proceedings of the DAC*.
- [112] W. Li, R. Purdy, J. M. F. Moura, and R. D. Blanton. 2023. Characterize the ability of GNNs in attacking logic locking. In *Proceedings of the MLCAD*.
- [113] Y. Li, A. Agnesina, Y. Zhang, H. Ren, and C. Yu. 2024. BoolGebra: Attributed graph-learning for boolean algebraic manipulation. In *Proceedings of the DATE*.
- [114] Y. Li, D. Tarlow, M. Brockschmidt, and R. S. Zemel. 2016. Gated graph sequence neural networks. In *Proceedings of the ICLR*.
- [115] R. Liang, H. Xiang, D. Pandey, L. Reddy, S. Ramji, G. J. Nam, and J. Hu. 2020. DRC hotspot prediction at sub-10nm process nodes using customized convolutional network. In *Proceedings of the ISPD*.
- [116] N. Limaye, S. Patnaik, and O. Sinanoglu. 2022. Valkyrie: Vulnerability assessment tool and attack for provably-secure logic locking techniques. *TIFS* 17 (2022), 744–759.
- [117] M. Liu, W. J. Turner, G. F. Kokai, B. Khailany, D. Z. Pan, and H. Ren. 2021. Parasitic-aware analog circuit sizing with graph neural networks and bayesian optimization. In *Proceedings of the DATE*.
- [118] S. Liu, W. Fang, Y. Lu, Q. Zhang, and Z. Xie. 2025. Towards big data in AI for EDA research: Generation of new pseudo circuits at RTL stage. In *Proceedings of the ASPDAC*. 228–251.
- [119] W. Liu, C. H. Chang, X. Wang, C. Liu, J. M. Fung, M. Ebrahimabadi, N. Karimi, X. Meng, and K. Basu. 2021. Two sides of the same coin: Boons and banes of machine learning in hardware security. *JETCAS* 11, 2 (2021).
- [120] Y. Liu, Z. Ju, Z. Li, M. Dong, H. Zhou, J. Wang, F. Yang, X. Zeng, and L. Shang. 2022. GraphPlanner: Floorplanning with graph neural network. *TODAES* 28, 2 (2022), 1–24.
- [121] Daniela Sánchez, Lorenzo Servadei, Gamze Naz Kiprit, Robert Wille, and Wolfgang Ecker. 2022. A comprehensive survey on electronic design automation and graph neural networks: Theory and applications. *TODAES* 28, 2 (2022), 1–27.
- [122] D. S. Lopera and W. Ecker. . Applying GNNs to timing estimation at RTL (invited paper). In *Proceedings of the ICCAD*.
- [123] D. S. Lopera, L. Servadei, G. N. Kiprit, S. Hazra, R. Wille, and W. Ecker. 2021. A survey of graph neural networks for electronic design automation. In *Proceedings of the MLCAD*.
- [124] L. Lu, J. Chen, M. Ulbricht, and M. Krstic. 2023. Towards critical flip-flop identification for soft-error tolerance with graph neural networks. *TCAD* 43, 4 (2023), 1135–1148.
- [125] Y. Lu, S. S. Kiran Pentapati, L. Zhu, K. Samadi, and S. K. Lim. 2020. TP-GNN: A graph neural network framework for tier partitioning in monolithic 3D ICs. In *Proceedings of the DAC*.
- [126] Y. Lu and S. K. Lim. . On advancing physical design using graph neural networks (invited paper). In *Proceedings of the ICCAD*.
- [127] Y. Lu, S. Nath, V. Khandelwal, and S. K. Lim. 2021. Doomed run prediction in physical design by exploiting sequential flow and graph learning. In *Proceedings of the ICCAD*.
- [128] Y. Lu, S. Nath, V. Khandelwal, and S. K. Lim. 2021. RL-Sizer: VLSI gate sizing for timing optimization using deep reinforcement learning. In *Proceedings of the DAC*.
- [129] Y. Lu, S. Pentapati, and S. K. Lim. VLSI placement optimization using graph neural networks. In *Proceedings of the NeurIPS*.
- [130] Y. Lu, S. Pentapati, and S. K. Lim. 2021. The law of attraction: Affinity-aware placement optimization using graph neural networks. In *Proceedings of the ISPD*.
- [131] Y. Ma, Z. He, W. Li, L. Zhang, and B. Yu. 2020. Understanding graphs in EDA: From shallow to deep learning. In *Proceedings of the ISPD*.
- [132] Y. Ma, H. Ren, B. Khailany, H. Sikka, L. Luo, K. Natarajan, and B. Yu. 2019. High performance graph convolutional networks with applications in testability analysis. In *Proceedings of the DAC*.
- [133] L. Mankali, L. Alrahis, S. Patnaik, J. Knechtel, and O. Sinanoglu. 2022. Titan: Security analysis of large-scale hardware obfuscation using graph neural networks. *TIFS* 18 (2022), 304–318.

- [134] I. L. Markov. 2024. The False Dawn: Reevaluating Google’s Reinforcement Learning for Chip Macro Placement. (2024). arXiv:2306.09633. Retrieved from <https://arxiv.org/abs/2306.09633>
- [135] I. McLoughlin. 2011. Reverse engineering of embedded consumer electronic systems. In *Proceedings of the ISCE*.
- [136] X. Meng, A. Sengupta, and K. Basu. 2023. A Needle in the Haystack: Inspecting Circuit Layout to Identify Hardware Trojans. *Cryptol. ePrint*. (2023).
- [137] A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. Songhori, S. Wang, Y. J. Lee, E. Johnson, O. Pathak, A. Nova, J. Pak, A. Tong, K. Srinivasa, W. Hang, E. Tuncer, Q. V. Le, J. Laudon, R. Ho, R. Carpenter, and Jeff Dean. 2021. A graph placement methodology for fast chip design. *Nature* 594, 7862 (2021), 207–212.
- [138] T. Mohamed, V. M. van Santen, L. Alrahis, O. Sinanoglu, and H. Amrouch. 2024. Graph attention networks to identify the impact of transistor degradation on circuit reliability. *TCAS-I* 71, 7 (2024), 3269–3281.
- [139] N. Muralidhar, N. Muralidhar, A. Zubair, N. Weidler, R. Gerdes, and N. Ramakrishnan. 2021. Contrastive graph convolutional networks for hardware trojan detection in third party IP cores. In *Proceedings of the HOST*.
- [140] W. Pan, M. Dong, C. Wen, H. Liu, S. Zhang, B. Shi, Z. Di, Z. Qiu, Y. Gao, and L. Zheng. 2023. A unioned graph neural network based hardware Trojan node detection. *IEICE Electron. Express* 20, 13 (2023), 20230204.
- [141] M. Patel, K. Jariwala, and C. Chattopadhyay. 2024. A systematic review on graph neural network-based methods for stock market forecasting. *ACM Computing Surveys* 57, 2, Article 34 (2024), 1–38.
- [142] S. Patnaik, M. Ashraf, H. Li, J. Knechtel, and O. Sinanoglu. 2021. Concerted wire lifting: Enabling secure and cost-effective split manufacturing. *TCAD* 41, 2 (2021), 266–280.
- [143] S. Patnaik, M. Ashraf, O. Sinanoglu, and J. Knechtel. 2020. Obfuscating the interconnects: Low-cost and resilient full-chip layout camouflaging. *TCAD* 39, 12 (2020), 4466–4481.
- [144] S. Patnaik, N. Rangarajan, J. Knechtel, O. Sinanoglu, and S. Rakheja. 2018. Advancing hardware security using polymorphic and stochastic spin-hall effect devices. In *Proceedings of the DATE*.
- [145] B. Perozzi, R. Al-Rfou, and S. Skiena. 2014. Deepwalk: Online learning of social representations. In *Proceedings of the KDD*.
- [146] L. Pouchet and T. Yuki. 2016. PolyBench/C - The Polyhedral Benchmark Suite. Retrieved from <http://web.cs.ucla.edu/~pouchet/software/polybench/>. (2016).
- [147] A. Rahimi, L. Benini, and R. K. Gupta. 2016. Variability mitigation in nanometer CMOS integrated systems: A survey of techniques from circuits to software. *Proceedings of the IEEE* 104, 7 (2016), 1410–1448.
- [148] F. Rahman, D. Forte, and M. M. Tehranipoor. 2016. Reliability vs. security: Challenges and opportunities for developing reliable and secure integrated circuits. In *Proceedings of the IRPS*.
- [149] J. Rajendran, M. Sam, O. Sinanoglu, and R. Karri. 2013. Security analysis of integrated circuit camouflaging. In *Proceedings of the CCS*.
- [150] J. Rajendran, O. Sinanoglu, and R. Karri. 2013. Is split manufacturing secure?. In *Proceedings of the DATE*.
- [151] J. Rajendran, O. Sinanoglu, and R. Karri. 2014. Regaining trust in VLSI design: Design-for-trust techniques. *Proceedings of the IEEE* 102, 8 (2014), 1266–1282.
- [152] S. G. Ramasubramanian, S. Venkataramani, A. Parandhaman, and A. Raghunathan. 2013. Relax-and-retchime: A methodology for energy-efficient recovery based design. In *Proceedings of the DAC*.
- [153] M. Randolph and W. Diehl. 2020. Power side-channel attack analysis: A review of 20 years of study for the layman. *Cryptography* 4, 2 (2020), 15.
- [154] M. Rapp, H. Amrouch, Y. Lin, B. Yu, D. Z. Pan, M. Wolf, and J. Henkel. 2022. MLCAD: A survey of research in machine learning for CAD keynote paper. *TCAD* 41, 10 (2022), 3162–3181.
- [155] B. Reagen, R. Adolf, P. Whatmough, G. Wei, and D. Brooks. 2014. MachSuite: Benchmarks for accelerator design and customized architectures. In *Proceedings of the IISWC*.
- [156] H. Ren, G. F. Kokai, W. J. Turner, and T. S. Ku. ParaGraph: Layout parasitics and device parameter prediction using graph neural networks. In *Proceedings of the DAC*.
- [157] H. Ren, S. Nath, Y. Zhang, H. Chen, and M. Liu. 2022. Why are graph neural networks effective for EDA problems? (invited paper). In *Proceedings of the ICCAD*.
- [158] M. Rostami, F. Koushanfar, and R. Karri. 2014. A primer on hardware security: Models, methods, and metrics. *Proceedings of the IEEE* 102, 8 (2014), 1283–1295.
- [159] M. Rostami, F. Koushanfar, J. Rajendran, and R. Karri. 2013. Hardware security: Threat models and metrics. In *Proceedings of the ICCAD*.
- [160] J. A. Roy, F. Koushanfar, and L. Igor. EPIC: Ending piracy of integrated circuits. In *Proceedings of the DATE*.
- [161] D. Saha and S. Sur-Kolay. 2011. SoC: A real platform for IP reuse, IP infringement, and IP protection. *VLSI Design* (2011).
- [162] D. Saha, J. Zhou, and F. Farahmandi. 2025. Physical Design-Aware Power Side-Channel Leakage Assessment Framework using Deep Learning. *Cryptol. ePrint*. 2011, 1 (2025), 731957.

- [163] A. Said, M. Shabbir, B. Broll, W. Abbas, P. Völgyesi, and X. Koutsoukos. 2023. Circuit design completion using graph neural networks. *Neural Computing and Applications* 35, 16 (2023), 12145–12157.
- [164] T. Sakurai and A. R. Newton. 1990. Alpha-power law MOSFET model and its applications to CMOS inverter delay and other formulas. *JSSC* 25, 2 (1990), 584–594.
- [165] L. Scheffer, L. Lavagno, and G. Martin. 2018. *EDA for IC system design, verification, and testing*. CRC Press.
- [166] A. Sengupta. 2016. Hardware security of CE devices [hardware matters]. *IEEE Consumer Electronics Magazine* 6, 1 (2016), 130–133.
- [167] A. Shahane, S. Swapna Manjiri, A. Jain, and S. Kumar. 2024. Graph of circuits with GNN for exploring the optimal design space. *NeurIPS* 36 (2024), 6014–6025.
- [168] K. Shamsi, M. Li, K. Plaks, S. Fazzari, D. Z. Pan, and Y. Jin. 2019. IP protection and supply chain security through logic obfuscation: A systematic overview. *TODAES* 24, 6 (2019), 1–36.
- [169] Z. Shi, M. Li, S. Khan, L. Wang, N. Wang, Y. Huang, and Q. Xu. 2022. Deeptpi: Test point insertion with deep reinforcement learning. In *Proceedings of the ITC*.
- [170] D. Sisejkovic, F. Merchant, L. M. Reimann, H. Srivastava, A. Hallawa, and R. Leupers. 2021. Challenging the security of logic locking schemes in the era of deep learning: A neuroevolutionary approach. *JETC* 17, 3, Article 30 (2021), 1–26.
- [171] D. Sisejkovic, F. Merchant, L. M. Reimann, and R. Leupers. 2021. Deceptive logic locking for hardware integrity protection against machine learning attacks. *TCAD* 41, 6 (2021), 1716–1729.
- [172] D. Sisejkovic, L. M. Reimann, E. Moussavi, F. Merchant, and R. Leupers. 2021. Logic locking at the frontiers of machine learning: A survey on developments and opportunities. In *Proceedings of the VLSI-SoC*.
- [173] J. Srinivasan, S. V. Adve, P. Bose, and J. A. Rivers. 2004. The case for lifetime reliability-aware microprocessors. *SIGARCH Computer Architecture News* 32, 2 (2004), 276.
- [174] A. Srivastava, S. Das, N. Choudhury, R. Psiakis, P. H. Silva, D. Pal, and K. Basu. 2024. SCAR: Power side-channel analysis at RTL level. *TVLSI* 32, 6 (2024), 1110–1123.
- [175] P. Subramanyan, N. Tsiskaridze, K. Pasricha, D. Reisman, A. Susnea, and S. Malik. 2013. Reverse engineering digital circuits using structural and functional analyses. *TETC* 2, 1 (2013), 63–80.
- [176] P. Subramanyan, S. Ray, and S. Malik. 2015. Evaluating the security of logic encryption algorithms. In *Proceedings of the HOST*.
- [177] Lichao Sun, Yingdong Dou, Carl Yang, Kai Zhang, Ji Wang, Philip S. Yu, Lifang He, and Bo Li. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering* 35, 8 (2022), 1–20. DOI: <https://doi.org/10.1109/tkde.2022.3201243>
- [178] K. Swaminathan, N. Chandramoorthy, C. Cher, R. Bertran, A. Buyuktosunoglu, and P. Bose. 2017. Bravo: Balanced reliability-aware voltage optimization. In *Proceedings of the HPCA*.
- [179] Synopsys. 2015. *Primetime User Guide*. Retrieved from http://www.synopsys.com/Tools/Implementation/SignOff/Documents/primetime_ds.pdf
- [180] A. F. Tabrizi, N. K. Darav, S. Xu, L. Rakai, I. Bustany, A. Kennings, and L. Behjat. 2018. A machine learning framework to identify detailed routing short violations from a placed netlist. In *Proceedings of the DAC*.
- [181] S. Takamaeda-Yamazaki. 2015. Pyverilog: A python-based hardware design processing toolkit for verilog HDL. In *Proceedings of the ARC*.
- [182] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei. 2015. Line: Large-scale information network embedding. In *Proceedings of the WWW*.
- [183] D. M. Tax and R. P. Duin. 2004. Support vector data description. *Machine Learning* 54, 1 (2004), 45–66.
- [184] E. Ustun, C. Deng, D. Pal, Z. Li, and Z. Zhang. 2020. Accurate operation delay prediction for FPGA HLS using graph neural networks. In *Proceedings of the ICCAD*.
- [185] A. Van den Oord, N. Kalchbrenner, L. Espeholt, O. Vinyals, A. Graves, and K. Kavukcuoglu. 2016. Conditional image generation with pixcnn decoders. *NeurIPS* 29 (2016), 4797–4805.
- [186] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio. 2018. Graph attention networks. In *Proceedings of the ICLR*.
- [187] S. Wallat, M. Fyrbiak, M. Schlögel, and C. Paar. 2017. A look at the dark side of hardware reverse engineering—a case study. In *Proceedings of the IVSW*.
- [188] Fangxin Wang, Yuqing Liu, Kay Liu, Yibo Wang, Sourav Medya, and Philip S. Yu. 2025. Uncertainty in Graph Neural Networks: A Survey. (2025). arXiv:2403.07185. Retrieved from <https://arxiv.org/abs/2403.07185>
- [189] H. Wang, D. Forte, M. M. Tehranipoor, and Q. Shi. 2017. Probing attacks on integrated circuits: Challenges and research opportunities. *IEEE Design & Test* 34, 5 (2017), 63–71.
- [190] H. Wang, K. Wang, J. Yang, L. Shen, N. Sun, H. Lee, and S. Han. 2020. GCN-RL circuit designer: Transferable transistor sizing with graph neural networks and reinforcement learning. In *Proceedings of the DAC*.
- [191] H. Wang, J. Yang, H. Lee, and S. Han. 2018. Learning to design circuits. (2018) arXiv:1812.02734. Retrieved from <https://arxiv.org/abs/1812.02734>

- [192] H. Wang, Z. Zhang, H. Xiong, D. Zou, Y. Chen, and H. Jin. 2024. GRAND: A graph neural network framework for improved diagnosis. *TCAD* 43, 4 (2024), 1288–1301.
- [193] L. Wang, Y. Chang, and K. Cheng. 2009. *Electronic Design Automation: Synthesis, Verification, and Test*. Morgan Kaufmann.
- [194] M. Wang, D. Zheng, Z. Ye, Q. Gan, M. Li, X. Song, J. Zhou, C. Ma, L. Yu, Y. Gai, T. Xiao, T. He, G. Karypis, J. Li, and Z. Zhang. 2020. Deep Graph Library: A Graph-Centric, Highly-Performant Package for Graph Neural Networks. (2020). arXiv:1909.01315. Retrieved from <https://arxiv.org/abs/1909.01315>
- [195] Z. Wang, C. Bai, Z. He, G. Zhang, Q. Xu, T. Y. Ho, B. Yu, and Y. Huang. 2022. Functionality matters in netlist representation learning. In *Proceedings of the DAC*.
- [196] Z. Wang, L. Alrahis, L. Mankali, J. Knechtel, and O. Sinanoglu. 2024. LLMs and the future of chip design: Unveiling security risks and building trust. In *Proceedings of the ISVLSI*.
- [197] Z. Wang, Z. He, C. Bai, H. Yang, and B. Yu. 2023. Efficient arithmetic block identification with graph learning and network-flow. *TCAD* 42, 8 (2023), 2591–2603.
- [198] H. Wilson. 2020. Heterogeneous GraphSAGE (HinSAGE). (2020). Retrieved from <https://github.com/stellargraph/stellargraph>
- [199] L. Wu, Y. Chen, K. Shen, X. Guo, H. Gao, S. Li, J. Pei, and B. Long. 2023. Graph neural networks for natural language processing: A survey. *Foundations and Trends in Machine Learning* 16, 2 (2023), 119–328.
- [200] N. Wu, J. Lee, Y. Xie, and C. Hao. 2022. Logic Optimization via Spatio-Temporal Information with Hybrid Graph Models. In *Proceedings of the ASAP*.
- [201] N. Wu, Y. Li, C. Hao, S. Dai, C. Yu, and Y. Xie. 2023. Gamora: Graph learning based symbolic reasoning for large-scale boolean networks. In *Proceedings of the DAC*.
- [202] N. Wu, Y. Xie, and C. Hao. 2021. IRONMAN: GNN-assisted design space exploration in high-level synthesis via reinforcement learning. In *Proceedings of the GLSVLSI*.
- [203] N. Wu, Y. Xie, and C. Hao. 2022. IronMan-Pro: Multi-objective design space exploration in HLS via reinforcement learning and graph neural network based modeling. *TCAD* 42, 3 (2022), 900–913.
- [204] N. Wu, H. Yang, Y. Xie, P. Li, and C. Hao. 2022. High-level synthesis performance prediction using GNNs: Benchmarking, modeling, and advancing. In *Proceedings of the DAC*.
- [205] S. Wu, F. Sun, W. Zhang, X. Xie, and B. Cui. 2022. Graph neural networks in recommender systems: A survey. *ACM Computing Surveys* 55, 5 (2022), 1–37.
- [206] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu. 2021. A comprehensive survey on graph neural networks. *TNNLS* 32, 1 (2021), 4–24.
- [207] Z. Xie, R. Liang, X. Xu, J. Hu, Y. Duan, and Y. Chen. 2021. Net2: A graph attention network method customized for pre-placement net length estimation. In *Proceedings of the ASP-DAC*.
- [208] K. Xu, W. Hu, J. Leskovec, and S. Jegelka. 2019. How powerful are graph neural networks?. In *Proceedings of the ICLR*.
- [209] Z. Xu. 2023. An intelligent fault detection approach for digital integrated circuits through graph neural networks. *Mathematical Biosciences and Engineering* 20, 6 (2023), 9992–10006.
- [210] S. Yang, Z. Yang, D. Li, Y. Zhang, Z. Zhang, G. Song, and J. HAO. 2022. Versatile multi-stage graph neural network for circuit representation. In *Proceedings of the NeurIPS*.
- [211] R. Yasaei, S. Faezi, and M. A. A. Faruque. 2022. Golden reference-free hardware trojan localization using graph convolutional network. *TVLSI* 30, 10 (2022), 1401–1411.
- [212] R. Yasaei, L. Chen, S. Yu, and M. A. A. Faruque. 2022. Hardware trojan detection using graph neural networks. *TCAD* 44, 1 (2022), 25–38.
- [213] R. Yasaei, S. Yu, and M. A. Al Faruque. 2021. GNN4TJ: Graph neural networks for hardware trojan detection at register transfer level. In *Proceedings of the DATE*.
- [214] R. Yasaei, S. Yu, E. K. Naeini, and M. A. A. Faruque. 2021. GNN4IP: Graph neural network for hardware intellectual property piracy detection. In *Proceedings of the DAC*.
- [215] M. Yasin, B. Mazumdar, O. Sinanoglu, and J. Rajendran. 2020. Removal attacks on logic locking and camouflaging techniques. *TETC* 8, 2 (2020), 517–532.
- [216] J. Ye, P. Ren, Y. Xue, H. Fang, and Z. Ji. 2023. Fast aging-aware timing analysis framework with temporal-spatial graph neural network. *TCAD* 43, 6 (2023), 1862–1871.
- [217] Y. Ye, T. Chen, Y. Gao, H. Yan, B. Yu, and L. Shi. 2023. Graph-learning-driven path-based timing analysis results predictor from graph-based timing analysis. In *Proceedings of the ASP-DAC*.
- [218] Z. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec. 2019. Gnnexplainer: Generating explanations for graph neural networks. *NeurIPS* 32 (2019), 9244–9255.
- [219] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen. 2020. Graph contrastive learning with augmentations. *NeurIPS* 33 (2020), 5812–5823.

- [220] S. Y. Yu, R. Yasaei, Q. Zhou, T. Nguyen, and M. A. A. Faruque. 2021. HW2VEC: A graph learning tool for automating hardware security. In *Proceedings of the HOST*.
- [221] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. Prasanna. 2019. GraphSAINT: Graph sampling based inductive learning method. In *Proceedings of the ICLR*.
- [222] P. Zhan, H. Shen, S. Li, and H. Li. 2023. BGNN-HT: Bidirectional graph neural network for hardware trojan cells detection at gate level. In *Proceedings of the ISCAS*.
- [223] G. Zhang, H. He, and D. Katabi. 2019. Circuit-GNN: Graph neural networks for distributed circuit design. In *Proceedings of the ICML*.
- [224] M. Zhang and Y. Chen. 2018. Link Prediction Based on Graph Neural Networks. In *Proceedings of the NIPS*.
- [225] M. Zhang, Z. Cui, M. Neumann, and Y. Chen. 2018. An end-to-end deep learning architecture for graph classification. In *Proceedings of the AAAI*.
- [226] M. Zhang, S. Jiang, Z. Cui, R. Garnett, and Y. Chen. 2019. D-vae: A variational autoencoder for directed acyclic graphs. *NeurIPS* 32 (2019), 1588–1600.
- [227] Y. Zhang, H. Ren, and B. Khailany. 2020. GRANNITE: Graph neural network inference for transferable power estimation. In *Proceedings of the DAC*.
- [228] Z. Zhang, X. Jiang, R. Wang, S. Guo, Y. Wang, and R. Huang. 2018. Extraction of process variation parameters in FinFET technology based on compact modeling and characterization. *TED* 65, 3 (2018), 847–854.
- [229] D. Zhao, S. Yuan, Y. Sun, S. Tu, and L. Xu. 2023. DeepTH: Chip placement with deep reinforcement learning using a three-head policy network. In *Proceedings of the DATE*.
- [230] G. Zhao and K. Shamsi. 2022. Graph neural network based netlist operator detection under circuit rewriting. In *Proceedings of the GLSVLSI*.
- [231] X. Zhou, J. Ye, C. W. Pui, K. Shao, G. Zhang, B. Wang, J. Hao, G. Chen, and P. A. Heng. 2022. Heterogeneous graph neural network-based imitation learning for gate sizing acceleration. In *Proceedings of the ICCAD*.
- [232] L. Zhu, Y. Gu, and X. Guo. 2023. RC-GNN: Fast and accurate signoff wire delay estimation with customized graph neural networks. In *Proceedings of the AICAS*.

Appendices

A Supplementary Materials: Appendix

A.1 Overview Tables

Table 5. Overview of Selected GNN for EDA Frameworks

GNN Task	Platform	GNN	#Layers	Loss Function	Graph Type	Circuit Type	Pooling	Readout	End-to-end Task
Node classification/ regression	[65]	GraphSAGE [70]	2	Squared error	Undirected	GLN	–	–	Congestion prediction
	Congestion-Net [97]	GAT [186]	16	MSE	Undirected	GLN	NA	NA	Routing congestion prediction
	GRANNITE [227]	Sequential GCN [96]	1	MSE	DGL	GLN	NA	NA	Transferable power estimation
	RC-GNN [232]	RC-GNN [232]	2	–	Heterogeneous	RC circuit	NA	NA	Wire delay prediction
	[48]	GraphSAGE	2	–	Directed	GLN	NA	NA	Short violation prediction
	[132]	GCN [96]	3	Cross-entropy	Directed	GLN	NA	NA	Observation point insertion
	GRAND [192]	GraphSAGE	2	Cross-entropy	Directed/ undirected/ bidirected	GLN	NA	NA	Diagnosis improvement
Edge classification/ regression	[106]	Multigraph GNN [198]	4	Log softmax	Bipartite multigraph	Analog circuit (transistor level netlist)	Max	NA	Analog circuit hierarchical annotation
	[122]	GCN [96] MPNN [66]	3	MSE	DAG	RTL	NA	NA	RTL timing analysis
	Net ² [207]	Custom [207]	3	–	Directed	GLN	NA	NA	Net length prediction
	D-SAGE [184]	Custom GraphSAGE [184]	2	Cross-entropy	DFG	HLS	NA	NA	FPGA operation delay prediction
	[68]	Custom [68]	3	Custom L_2	Heterogeneous	GLN	NA	NA	Pre-routing timing prediction

(Continued)

Table 5. Continued

GNN Task	Platform	GNN	#Layers	Loss Function	Graph Type	Circuit Type	Pooling	Readout	End-to-end Task
Graph regression	[61]	MPNN [66]	—	—	Hybrid CDFG	HLS C/C++ program	Sum/Mean	NA	HLS DSE
	BoolGebra [113]	GraphSAGE	3	MSE	AIG	GLN	Mean	—	Boolean algebraic manipulation
	[109]	—	3	—	—	—	NA	NA	Design block placement prediction
	DoomedRun [127]	GraphSAGE [70]	2	MSE	—	GLN	Mean	LSTM	Design viability prediction
	Circuit-GNN [223]	Custom [223]	—	L_1	—	Resonant circuit	Selects two nodes conn. to ports	Concat	Distributed circuit design
Graph generation	GraphPlanner [120]	Variational GCN	2	Reconstruction loss + KL divergence	Weighted undirected hypergraph	GLN	NA	NA	Chip floorplanning optimization
Graph clustering	GraphClus-Net [76]	Hierarchical GCN	3	N-cut based	Undirected	GLN	NA	NA	Circuit netlist partitioning
GNN-RL	[5, 6]	GraphSAGE [70]	—	Unsupervised	Directed	GLN	—	Shallow aggregation	Deep RL-based VLSI parameter optimisation
	IronMan [202, 203]	GCN [96]	1–4	Mean squared logarithmic; MAE	DFG	HLS C/C++ Program	NA	Mean	HLS DSE
	Edge-GNN [137]	GCN [96]	1	MSE	Hypergraph	GLN	NA	Reduce mean	Edge-based chip floorplanning
	DeepTH [229]	—	—	Custom [229]	Hypergraph	GLN	NA	NA	Chip placement
	DeepPR [51]	GCN [96]	2	MSE	Hypergraph	GLN	NA	NA	RL-based macro placement
	[42]	GraphSAGE	5	Custom [42]	Heterogeneous	GLN	Max	NA	Circuit Routing
	[128]	GraphSAGE [70]	2	Unsupervised	Undirected clique	GLN	NA	NA	Timing optimization
	Circuit Designer [190]	GCN [96]	7	Custom [190]	Undirected	Transistor	NA	NA	Transistor sizing
Representation learning	DeepGate [111]	DAG-GNN	2	L_1	Directed AIG	RTL/GLN	NA	Attention aggregation	Circuit representation
	[210]	Custom [210]	3	—	Heterogeneous	GLN	Max	Task-adaptive concat./Mean	Circuit representation
	DeepSeq [93]	DAG-GNN	3	L_1	DAG	GLN	NA	NA	Circuit representation
	SyncTREE [78]	Custom [78]	4/8/16/32/64	TContrast [78]	RC-Tree	Resistor Capacitor (RC) circuit	Global pooling	NA	Timing Prediction
	[43]	GGNN [114]	2	Cross-entropy	Heterogeneous multigraph	GLN	NA	NA	Symmetry constraint generation
	CktGNN [57]	Custom [57]	—	VAE	DAG	Analog circuit	Graph	NA	Analog circuit encoding
	GCX [167]	GCN, GraphSAGE, GAT	—	Custom [167]	Homogeneous	VLSI Circuits (analog/mixed signal)	NA	NA	Optimal DSE
Imitation learning	[231]	HGNN [83]	3–6	Weighted Cross-entropy	Heterogeneous directed	GLN	NA	NA	Gate sizing optimization

—means unspecified. NA means not applicable.

Table 6. Overview of Selected GNN for Hardware Reliability Frameworks

GNN Task	Platform	GNN	#Layers	Loss Function	Graph Type	Circuit Type	Pooling	Readout	End-to-end Task
Node classification/ regression	[46]	H-GCN [46]	2	MSE	Heterogeneous directed	Post-layout TNL	NA	NA	Aging prediction
	Deep H-GCN [47]	H-GCN [47]	2–8	MSE	Heterogeneous directed	Post-layout TNL	NA	NA	Aging prediction
	[209]	K-Means-GCN	3	–	Directed	Analog circuit	NA	NA	Fault diagnosis
	[124]	SAGE-GCN, GraphSAGE	3	Cross-entropy	Directed	GLN	NA	NA	Critical FF Identification
Subgraph/ graph regression	GNN4REL [16]	PNA [54]	4	MSE	Directed	GLN	NA	Add	Aging prediction
	[33]	GCN	2	–	Directed	GLN	NA	NA	FDR prediction
	[216]	GTU [185], GraphSAGE	3	R ² -score	Homogenous bidirectional	GLN	NA	Mean	Aging prediction
	[103]	GCN, GAT	8	MSE	Attention-enabled graphs	HLS (in C)	NA	Max	Fault Injection Evaluation

–means unspecified. NA means not applicable.

Table 7. Overview of Selected GNN for Hardware Security Frameworks

GNN Task	Platform	GNN	#Layers	Loss Function	Graph Type	Circuit Type	Pooling	Readout	End-to-end Task
Node classification	GateDet [50]	BiGCN [39]	1–6	Weighted Cross-entropy	Directed	GLN	NA	NA	HT detection
	[140]	Customized GraphSAGE	3	Cross-entropy	Directed	GLN	NA	NA	HT detection
	BGNN-HT [222]	Bidirectional GNN (BGNN)	3×2	Weighted Cross-entropy	Directed	GLN	NA	NA	HT detection
	[136]	GraphSAINT	–	–	Undirected	RTL/GLN	NA	NA	HT detection
	GNN-RE [13]	GraphSAINT [221]	5	Cross-entropy	Undirected	GLN	Multi-head attention	NA	Functional RE
	AppGNN [41]	GraphSAINT [221]	5	Cross-entropy	Undirected	GLN	Multi-head attention	NA	Functional RE
	ABGNN [73]	ABGNN [73]	1–5–2	Cross-entropy	Directed	GLN	NA	NA	Functional RE
	GAMORA [201]	GraphSAGE	4,8	NLL	AIGs	GLN	NA	NA	Functional RE
	[230]	GCN [96]	3	–	DAG	GLN	NA	NA	Functional RE
	RelGNN [53]	GraphSAGE [70]	3	NLL	–	GLN	NA	NA	Functional RE
	SCAR [174]	GCN	2	-	CDFG	RTL	NA	NA	Side-channel attack
	GNNUnlock [11]	GraphSAINT [221]	2	Cross-entropy	Undirected	GLN	NA	NA	DfTr attack
	GNNUnlock+ [10]	GraphSAINT [221]	2	Cross-entropy	Undirected	GLN	NA	NA	DfTr attack
	NetlistGNN [112]	NetlistGNN [112]	–	–	Undirected, Directed, and heterogeneous	GLN	NA	NA	DfTr attack
Subgraph/ graph regression/ classification	ICNet [49]	GAT [186]	2	MSE	Undirected	GLN	Attention	Attention	DfTr attack
	NCL [195]	FGNN [195]	–	Cross-entropy	Directed	GLN	NA	Mean	Functional RE
	GNN4IJ [213]	GCN [96]	2	Cross-entropy	DFG	RTL/GLN	Top-k filtering	Max	HT detection
	GNN4IP [214]	GCN [96]	2	Cosine-similarity	DFG	RTL/GLN	Top-k filtering	Max	IP piracy
	[72]	GCN	2	Cross-entropy	Directed	GLN	NA	Mean	HT detection
	[77]	GCN	2	Cross-entropy	Directed	GLN	NA	Mean	IP piracy
	[75]	GCN/GAT/GIN	2	SVDD [183]	Undirected	GLN	NA	Mean	IP piracy
	[162]	GCN	2	-	-	RTL	Global Avg.	NA	Side-channel attack
	DepthGraphNet [26]	SGNN [26]	<29	Binary Cross-entropy	Directed	GLN	Global Sum	NA	Functional RE
	PoisonedGNN [15]	GCN	2	Cross-entropy/ Cosine-similarity	DFG	RTL/GLN	Top-k filtering	Max	HT/IP piracy
	OMLA [14]	GIN [208]	5–6	Cross-entropy	Undirected	GLN	Summing node features from the same layer	Sum	DfTr attack
	Titan [133]	GIN [208]	6	Cross-entropy	Undirected	GLN	Summing node features from the same layer	Sum	DfTr attack
Representation learning	GATE-Net [139]	GCN	3	Supervised Contrastive	DAG	GLN	NA	Mean	HT detection
	ConVERTS [52]	AIGNet [52]	–	Contrastive	AIG	RTL	NA	Concat. Mean and Max	Functional RE
Link prediction	MuxLink [22]	DGCNN [225]	4	Cross-entropy	Undirected	GLN	DGCNN sort-pooling [225]	Max	DfTr attack
	UNTANGLE [20]	DGCNN [225]	4	Cross-entropy	Undirected	GLN	DGCNN sort-pooling [225]	Max	DfTr attack

– means unspecified, NA means not applicable.

A.2 Feature Tables

Table 8. Taxonomy of Feature Elements for GNN in EDA

Major Category	Subcategory	Feature Elements	Paper List
Topology and Connectivity	Node Connectivity	(1) Fanin/Fanout; (2) Number of 1-hop and 2-hop neighbors; (3) Number of nodes; (4) Number of strongly connected components; (5) Circuit connectivity, Clique; (6) Primary Input/Output (PI/PO); (7) Clustering information, Cluster ID, Clustering coefficient; (8) Rich Club Coefficient;	[207], [231], [111], [137], [137], [207], [202, 203], [5, 6], [122], [68], [232], [120], [125], [109], [106], and [78]
	Hierarchy	(9) Hierarchy and memory features; (10) Depth level, PI/PO Depth; (11) Logic level	[5, 6], [122], [125], [129, 130], [132], and [192]
	Path Information	(12) Shortest path to clock; (13) Path and segment information	[125], [129, 130]
Physical Features	Dimensions/Layout	(14) Width, Height; (15) Cell size, Block area; (16) Coordinates (x,y), Location; (17) Relative position, Placement status/order; (18) Geometry (length, width); (19) Footprint; (20) Angular position of slit	[207], [231], [65], [97], [111], [57], [137], [68], [65], [76], [109], [229], [43], and [48]
	Density	(21) Pin density; (22) Pin count/number; (23) Cell, Macro, Blockage density; (24) Net density; (25) Number of grids; (26) Density constraints or limits	[65], [97], [210], [137], [65], [120], and [48]
	Layers/Routing	(27) Number of metal layers; (28) Metal layer; (29) Via number/count; (30) Wire details; (31) Physical wirelength	[137], [120], [128], [43], and [48]
Electrical Features	Component Parameters	(32) Capacitance; (33) Equivalent node load; (34) Resistance; (35) Transconduct; (36) Transistor parameters (widths, lengths, V_{sat} , V_{th0} , V_{fb} , μ_0 , U_C); (37) Load resistance; (38) Reference current/voltage	[57], [231], [190], [68], [232], [128], [78], [167]
	Timing Features	(39) Delay, RC delay; (40) State probabilities; (41) Transition probability/correlation; (42) Slew (Input slew, Output slew); (43) Slack, Worst slack	[231], [127], [122], [227], [125], and [128]
	Power Features	(44) Power (Switching, Internal, Leakage); (45) Logical net power	[127], [48]
Logical Features	Node Types	(46) Gate/Cell type; (47) Node type/attribute; (48) Device type, Vertex type (NMOS, PMOS, Resistor, Inductor, Capacitor, Net); (49) Net type (Input, output, bias, enable, io port, antenna, clock, power, ground); (50) RTL node type, Operation type	[97], [111], [93], [223], [137], [51], [202, 203], [190], [122], [232], [76], [184], [43], and [106]
	Edge Features	(51) Edge type (control, dataflow, parameter); (52) Edge properties (resistance, validity); (53) Signal characteristics (transition correlation); (54) Physical edge properties (wire resistance)	[223], [51], [122], [68], [227], [232], [113], [78]
	Data Characteristics	(55) Bitwidth, Data precision; (56) LUT indices; (57) Skip connections; (58) K-Colorability; (59) Spectral characteristics (Fiedler value, spectral radius)	[111], [202, 203], [5, 6], [122], [68], and [184]
Design and Optimization	Design Constraints	(60) Constraints details; (61) Design rule violations; (62) Controllability, Observability; (63) Lambda, Delay Lambda	[231], [132]
	Optimization Parameters	(64) Gain, Sizable options; (65) Transformability (rewriting/refactoring); (66) HLS Pragma values, HLS directive;	[231], [223], [202, 203], and [113]
Diagnostic Features	Flags/Counters	(67) Candidate flag; (68) Consistent/Inconsistent number; (69) Dependency level/number; (70) Overflow indicator; (71) Via and terminal segment indicators	[42], [48], and [192]
	Normalized Features	(72) Normalized length, segment, via count; (73) Normalized coordinates; (74) Normalized layer indices; (75) Normalized path resistance, design rule violations; (76) Normalized history costs; (77) Normalized avg./std. deviation/max difference of path resistances	[42]

Table 9. Taxonomy of Feature Elements for GNN in Hardware Reliability

Major Category	Subcategory	Feature Elements	Paper List
Topology and Connectivity	Node Connectivity	(1) Number of node loads by gate; (2) Number of connections to PI/PO; (3) Connections (source, drain and gate); (4) Clustering information; (5) Probabilistic graph embeddings	[209], [16], [124], [216], and [103]
Electrical Features	Signal Characteristics	(6) Input coverage; (7) Output frequency; (8) Ratio of positive value; (9) Min/Max toggling interval	[124]
	Component Parameters	(10) Design parameters (MOS, MOS spice, DIO/ESD, Cap., R, VSource); (11) Dynamic stress biases (V_{dt} , V_{gs}); (12) Static device features (gate width and length)	[46], [47], and [216]
Logical Features	Node Types	(13) Node type; (14) Gate types/orders;	[16], [103], [124], and [216]

Table 10. Taxonomy of Feature Elements for GNN in Hardware Security

Major Category	Subcategory	Feature Elements	Paper List
Topology and Connectivity	Node Connectivity	(1) In-degree/Out-degree; (2) Circuit connectivity; (3) Cell neighborhood; (4) Primary PI/PO; (5) Betweenness/Harmonic centrality	[72], [50], [140], [136], [75], [13], [41], [26], [230], [53], [11], [10], [14], [22], [133], [174], and [162]
	Hierarchy/Path Info.	(6) Minimum distance to PI/POs; (7) Distance encoding	[140], [14], [22], [20], and [133]
Logical Features	Node Types	(8) Node/Gate/Cell type (PI/PO/intermediate/operation); (9) Port connection of flip-flops; (10) Key-gate mask	[213], [15], [72], [50], [140], [222], [136], [139], [214], [77], [75], [13], [41], [26], [73], [201], [230], [195], [53], [52], [49], [11], [10], [14], [22], [20], [133], and [112]
	Edge Features	(11) Edge direction/acyclic properties; (12) Signal probability	[73], [201], [230], and [112]
	Data Characteristics	(13) Bit-flip data under normal/overclocked conditions; (14) Truth table; (15) NPN class;	[72], [230]

A.3 GNN4CIRCUITS: Implementation Details

A.3.1 Gate-level netlist parsing. The parser processes the user-provided technology library to extract module types and their ports into a dictionary. It then parses GLN files and constructs NetworkX [69] graphs connecting nodes based on gate type and port mappings. Ports from the top module are categorized as inputs, outputs, or wires.

GNN4CIRCUITS parses bench-formatted circuit descriptions using a parser adapted from Mux-Link [22] and modified to make it more general. It builds a directed NetworkX [69] graph, annotating nodes with gate type, instance count, and I/O status, and labels the graph using the directory name.

A.3.2 RTL parsing. For RTL Verilog parsing, we use Pyverilog [181] to transform the hardware design into a DFG. This DFG is converted into a NetworkX [69] graph suitable for subsequent pipeline steps. We adopted the RTL to DFG conversion portion as done by HW2VEC [220]

A.3.3 Text file parsing. For users who have an alternate way of extracting circuits, GNN4CIRCUITS supports parsing from specific text files: *cell.txt*, *label.txt*, *feat.txt*, *row.txt*, and *col.txt*, with formatting details provided.

A.3.4 Node Features. The optional node features supported by the platform include: input and output degrees, combined I/O degrees, minimum distances to primary inputs and outputs, boolean flags for primary input/output status, gate types and key input. NetworkX [69] calculates degrees and paths. Primary inputs/outputs are identified via the parsing dictionary. Gate types from netlists are one-hot encoded, and key inputs are flagged based on whether there is an input node which matches the key string provided.

A.3.5 Serializing the Graphs. Graphs are serialized into CSV format for consistent ML data handling. Graph-level labels correspond to parent directory names; node-level labels require a *label.txt* file. A *files4training* directory contains:

- *graph_edges.csv* – edges per graph by ID
- *graph_properties.csv* – graph ID, label (for graph-level tasks), node count, original string labels
- *node_features.csv* – node ID, features, labels (for node-level tasks) per graph ID

A.4 GNN4CIRCUITS: ML Pipeline

GNN4CIRCUITS leverages DGL [194] to enable user-generated datasets and provides multiple GNN architectures, along with structured setups for model training, validation, and testing.

A.4.1 Dataset Creation. The GraphDataset and NodeDataset classes load node features, edges and graph properties from CSV files to construct DGL [194] graphs, associating each graph with its respective graph or node labels and properties for GNN usage.

A.4.2 Model Developing and Testing. *GNN4CIRCUITS* includes a detailed setup for training, validating, and testing the models using DGL's `GraphDataLoader` [194] to handle batches of graph data. Early stopping is implemented based on validation loss. The DGL library [194] is used to develop the supported models, including: GCN [96], GIN [208], and PNA [54]. Test evaluations yield metrics (e.g., accuracy, precision, recall, F1, and MAE) determined by whether the task is classification or regression, and log statistics such as confusion matrices and runtime performance.

Received 4 November 2024; revised 9 July 2025; accepted 29 August 2025